NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

DIOGO HENRIQUE SILVESTRE MATOS CEBOLA

SENTRI: A SEARCHABLE ENCRYPTED TRIPLESTORE FOR SEMANTIC DATA

DEVELOPING PROTOCOLS TO SECURE EXPRESSIVE SPARQL QUERIES OVER ENCRYPTED DATA

MASTER IN COMPUTER SCIENCE

NOVA University Lisbon

Draft: *October 15, 2023*

NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

DEPARTMENT OF
COMPUTER SCIENCE

SENTRI: A SEARCHABLE ENCRYPTED TRIPLESTORE FOR SEMANTIC DATA

DEVELOPING PROTOCOLS TO SECURE EXPRESSIVE SPARQL QUERIES OVER ENCRYPTED DATA

DIOGO HENRIQUE SILVESTRE MATOS CEBOLA

Advisers: Matthias Knorr
Assistant Professor, NOVA University Lisbon
João Leitão
Assistant Professor, NOVA University Lisbon
Henrique Domingos
Associate Professor, NOVA University Lisbon

MASTER IN COMPUTER SCIENCE

NOVA University Lisbon
Draft: *October 15, 2023*

Abstract

Ontologies provide a shared understanding of a domain and, in general, only contain public, and non-sensitive data, but sometimes they are used alongside electronic health records and clinical decision systems. These systems heavily manipulate personal data, which is sensitive. Additionally, ontologies themselves have an inherent private value, due to the investment of time and money in their development.

With ontologies gradually growing in dimension, the industry is currently evolving towards the use of collaborative solutions and the outsourcing of heavy computations, relying on cloud infrastructures or other shared computing platforms. These approaches, albeit inevitable, present new security concerns.

In this thesis, we investigate existing secure computation techniques, encryption protocols and storage systems with the finality of developing a practical and secure solution for the collaborative development, distribution and usage of ontologies.

Our main contributions from this research are: (i) SEnTri, a searchable encrypted triplestore that can evaluate secure SPARQL read and write queries, possesses reasoning capabilities and supports access control; (ii) Two secure protocols for the evaluation of SPARQL queries, based on Searchable Symmetric Encryption techniques and secure equality joins; (iii) A novel secure equality join, inspired by the DGK cryptosystem, that operates over homomorphically encrypted values.

The system has been evaluated using the LUBM benchmark under different configurations with varying security and performance trade-offs. T[1]

1) *TODO:* Summarize experimental evaluation results

Keywords: Ontologies, Searchable Encryption, Secure Computing, SPARQL, Secure Equality Join

Resumo

As ontologias fornecem conhecimento partilhado referente a um domínio. Fazem-no através da especificação dos conceitos relevantes, e suas relações, num dado domínio.

Em geral, contêm apenas dados públicos e não sensíveis mas, por vezes, podem ser usadas em conjunto de sistemas de apoio à decisão clínica e repositórios de dados médicos. Nestes sistemas existe manipulação frequente de dados pessoais, de carácter sensível.

Atualmente, com o gradual aumento da dimensão das ontologia, a indústria está evoluir no sentido da utilização de ferramentas colaborativas e *outsourcing* de computações pesadas, recorrendo a infrastruturas *cloud* e outras plataformas de computação partilhada. Estas soluções, embora inevitáveis, apresentam novas questões de segurança.

Nesta tese, investigamos técnicas existentes de computação seguras e propomos a definição de um novo protocolo de Encriptação Simétrica Pesquisável, que suporta *queries* SPARQL expressivas sobre ontologias encriptadas.

Palavras-chave: Ontologias, Encriptação Pesquisável, Computação Segura, SPARQL

Contents

List of Figures

List of Listings

Acronyms

AES Advanced Encryption Standard 26
API Application Programming Interface 19

BGP Basic Graph Pattern 16
BSBM Berlin SPARQL Benchmark 19

DBMS Database Management Systems 26, 27

FHE Fully Homomorphic Encryption 3, 20

GO Gene Ontology 1
GOOSE Graph OutsOurcing and SPARQL Evaluation v, 4, 28

HE Homomorphic Encryption 2, 3, 6, 20, 29
HTML HyperText Markup Language 9
HTTPS Hyper Text Transfer Protocol Secure 31

IND-CKA2 Indistinguishability Against Adaptive Chosen Keyword Attack 23, 24, 30
IND-CKA1 Indistinguishability Against Non-Adaptive Chosen Keyword Attack 23, 24, 30
IND-CPA Indistinguishability Against Chosen Plaintext Attacks 23
IRI Internationalized Resource Identifier 6, 8, 12, 16

LFHE Leveled-Fully Homomorphic Encryption 3, 21
LUBM Lehigh University Benchmark 19, 32

M/M Multiple Writer/Multiple Reader 22, 31
M/S Multiple Writer/Single Reader 22

NCI National Cancer Institute 1

OPE Order-Preserving Encryption 21, 28
ORAM Oblivious RAM 3, 20, 29

OWL2 Web Ontology Language vi, vii, 1, 7, 11, 12, 13, 14, 15, 16, 19, 25, 29, 30, 32

PEKS Public Key Encryption with Keyword Searchable 3, 21, 29
PHE Partially Homomorphic Encryption 3, 20
PRF Pseudo-Random Function 23
PRP Pseudo-Random Permutation 24

RDF Resource Description Language vii, ix, 1, 7, 8, 9, 11, 16, 17, 18, 19, 28, 29
RDFS Resource Description Framework Schema 1, 7, 9, 10, 11, 12, 13, 29

S/M Single Writer/Multiple Reader 22
S/S Single Writer/Single Reader 22, 29, 31
SE Searchable Encryption iv, 2, 3, 6, 20, 21, 22, 23, 28, 29, 31
SHE Somewhat Homomorphic Encryption 3, 20
SLA Service Level Agreement 2
SPARQL SPARQL Protocol and RDF Query Language vii, 1, 3, 4, 5, 7, 16, 17, 18, 19, 29, 30, 32
SQL Structured Query Language 17, 26, 27, 28
SSE Symmetric Searchable Encryption v, 3, 4, 21, 23, 25, 29, 30, 33
STE Structured Symmetric Encryption 3, 25
SWRL Semantic Web Rule Language 7

TCP Transmission Control Protocol 31
TLS Transport Layer Security 31
TURTLE Terse RDF Triple Language vii, 8, 9, 13, 18

UCRPQ Unions of Conjunctions of Regular Path Queries 4, 28

W3C World Wide Web Consortium 6
WWW World Wide Web 1

XML Extensible Markup Language 7, 9, 11

1

Introduction

1.1 Context

The Semantic Web In 2001, the term *Semantic Web* [7] was first used to define what would be the evolution of the World Wide Web (WWW). The traditional *Web* solved the challenge of distributing information at scale and globally, however it was designed for human use. Documents are easily accessible and readable by humans, but for automated agents little more than document retrieval is possible. Contrary to humans, machines cannot reason over the information they access, if they do not have access to the *semantics of the data*.

The *Semantic Web* addresses this issue, by redesigning the representation of information, in a way that not only preserves meaning and relationships but is also machine-readable. A concept is mapped to a unique identifier, and its meaning is encoded in sets of triples, each triple consisting of a subject, predicate and object (similar to an elementary sentence). *Ontologies* provide a shared understanding of a domain, by formalizing the specification of relevant concepts, and their relations, in such domain.

This novel method of knowledge representation differs from the previous, centralized way, and greatly simplifies the process of interconnecting different knowledge bases. Centralized representations tend to be more expensive to develop, usually needing to be built from scratch [57]. Furthermore, for sharing, both parties need to agree on the semantics. In contrast, using ontologies, information is semantically linked by default. In the *Semantic Web*, machines can truly communicate, share and reason about distinct knowledge bases.

Semantic data is specified using a stack of different language specifications. The basic data model is specified using RDF [102], these resources can be organized in hierarchies with Resource Description Framework Schema (RDFS) [106] and to define ontologies we use OWL2 [95]. To query semantic data, the standard language is SPARQL [110, 111, 112].

Industry Landscape Knowledge representation is essential in the industry [60] and ontologies help remove ambiguity in terminology. Nowadays, they are being developed and applied in a variety of domains [108] ranging from the health sector[1], to linguistics [2]

[1] The NCI Thesaurus [79], developed by the National Cancer Institute (NCI) or the Gene Ontology (GO) [37], the world's largest source of information on the functions of genes, are some existing applications.

[2] The WordNet [53] is a lexical database for English where related words and concepts are semantically linked.

and even being used by companies like NASA[3] [6].

To developed and maintain ontologies, advanced semantic editors are the standard tool for professionals, with Protégé [56] being a very popular example. These provide numerous functionalities, usually based on plugins (e.g. automatic verification of errors and consistency in the design; debugging tools; query answering; graphic visualization of data and knowledge).

With datasets gradually growing in dimension, the industry is currently evolving towards the use of collaborative solutions [91, 90] and outsourcing heavy computations, relying on cloud infrastructures or other shared computing platforms. These approaches, albeit inevitable, present new security concerns and engineering challenges.

By 2021, 50 percent of all enterprise workloads were already hosted in *public clouds* [78]. As of 2023, worldwide end-user spending on public cloud services is forecast to grow 20.7% more than the growth forecast for 2022 of 18.8% [32].

Research on secure cloud computing [15] studies such adoption but also discusses security concerns. Adoption can be explained with the cost efficiency[4], high scalability, flexibility and availability that cloud provides. Additionally, the pay-per-use model is an easy entry point to the ecosystem.

Cloud Security Concerns While delegation of responsibilities is beneficial for costs, it is prejudicial to control. Using cloud services implies trusting its owner. A Service Level Agreement (SLA) defines, in accordance to metrics, the service levels of the system that the vendor should provide (e.g. percentage of availability), while also defining penalties in case of not achieving what was agreed-on. However, the risk of loss of privacy and data breaches still exists. A problem that is especially concerning when sensitive data is being stored by such services. This is much more relevant in *public clouds* where, compared to *private clouds*, trust is distributed between various actors and not on a single organization, namely the cloud operator [15].

In cloud storage services, administrators, or hacker with root privileges, might have access to the data [11]. Additionally, these cloud services might outsource computations to third parties whom we do not trust. In this setting, we usually consider the *honest-but-curious*, or *passive*, adversary model. This adversary will follow the protocol properly but keep a record of all its intermediate computations [34]. Furthermore, it cannot do any other action except attempting to learn private information while observing views of the protocol execution [26].

Security Approaches A first approach to secure data in the cloud would be to use *Encryption at Rest*, which naively encrypts all data stored in the untrusted servers. If a user wants to update or query the data, they will have to download all the data, and decrypt it. This incurs in high network requirements and is computationally expensive on the client-side, hindering all the benefits of using a cloud service to store the data.

Although we can observe some major database services, like MongoDB, starting to adopt Searchable Encryption [55], the majority of cloud and storage providers only support the former solution.

Currently, active research is being done in the fields of SE and Homomorphic Encryption (HE).

[3] Observing the visual representation provided by the Linked Open Data Cloud [51] is a good exercise to perceive the dimension of adoption of this new paradigm.

[4] Cloud services reduce implementation, maintenance, power, cooling, floor space, storage and operational costs.

Homomorphic Encryption is based on the concept of algebra homomorphisms, where both decryption and encryption can be seen as an homomorphism of the relations between the plaintext and the ciphertext [3, 31]. Homomorphic Encryption, based on the type and number of operations supported (e.g. addition or multiplication of encrypted values), can be classified as: Fully Homomorphic Encryption (FHE) supporting any number of operations with no restriction on the number of times; Somewhat Homomorphic Encryption (SHE) permitting some types of operations, a limited number of times; Partially Homomorphic Encryption (PHE) allowing one type of operation with no restriction on the number of times; and Leveled-Fully Homomorphic Encryption (LFHE) [12] supporting any type of operations, but with a pre-determined expected depth-L (can be executed up to L times).

Searchable Encryption uses classical encryption schemes to perform encrypted search queries, sequentially over the ciphertext [81], or through the generation of searchable *encrypted indexes* accessible with *tokens*, generated via *trapdoor* functions. Searchable Encryption can be divided in SSE, when it is based on symmetric key cryptography, and Public Key Encryption with Keyword Searchable (PEKS), when asymmetric key algorithms are used. These techniques may leak some information about the *indexes*, *search pattern*, and *access pattern* during execution. Most approaches leak, at least, the *search* and *access patterns* [11]. Recently, to mitigate information leakage, *hybrid* solutions combining the software approach of SE with *trusted hardware* have been proposed [29].

Besides HE and SE, Oblivious RAM (ORAM) [35] could theoretically be used as an alternative. However, ORAM and HE continue to be too inefficient for real world applications, when considered as an exclusive solution [11]. Furthermore, SSE is often more practical than PEKS, as asymmetric cryptographic schemes tend to be more efficient than their symmetric counterparts [66].

1.2 Problem

Usually, ontologies only contain public, and non-sensitive data, but sometimes they are used alongside electronic health records and clinical decision systems. These systems heavily manipulate personal data, which is sensitive. Additionally, ontologies themselves have an inherent private value, due to the investment of time and money in their development. With all these potential security needs, a solution that supports the secure development, sharing and usage of ontologies alongside sensitive data is essential.

Taking into consideration the efficiency, security trade-offs and ease of implementation from the various alternative techniques, a pure software SSE approach seems to be the most practical and easier to adopt solution, not requiring excessive computational power or special trusted hardware. Recently, Chase and Kamara, on their work on Structured Symmetric Encryption (STE), generalized index-based SSE to support queries on *structured* data [16].

Semantic data can be represented as a graph and STE supports various types of query operations in graph-structured data. Some existing schemes, adapted to conceptual graphs, need to pre-compute all possible query answers *a priori*, which is not practical for large datasets [70]. Furthermore, there can be leakage of information about the topology of the data while trying to pre-compute all query answers.

Recent developments do not require pre-computation of all query answers. The *CryptGraphDB* [2] system is an adaptation of CryptDB [73, 72, 71] capable of answering queries over conceptual graphs, but it was developed for the cypher [59] language and not SPARQL.

Another work, focused on access control for RDF datasets, did also implement a solution for retrieving simple triple patterns over an encrypted dataset, but, due to performance needs, hashes were used for indexing the encrypted triples. This leaks the structure of the dataset, which they tried to mitigate by creating dummy triples [28].

The GOOSE [18] framework, demonstrates how to use multiparty computation techniques to answer Unions of Conjunctions of Regular Path Queries (UCRPQ) over encrypted graphs, only relying on a standard SPARQL engine. But deterministic encryption is used, and therefore the structure of the graph is also leaked. This framework only focuses on SPARQL read queries.

The HDT_{crypt} [27] solution combines RDF compression with encryption. It is capable of efficient storage, but can only perform simple triple pattern search (joins and other complex operations need to be performed by the client).

Although, a lot of progress has been made in this field of research there are still various open questions. In this thesis, we will focus on answering the following questions:

1. *How to support secure and expressive read and write SPARQL queries over encrypted data?*

   A practical solution ought to support more than basic triple pattern retrieval, it must be capable of querying complex patterns. As such it should be able to execute secure set operations without the client needing to decrypt the intermediate values (obtained from simple triple patterns). Furthermore, it must be able to execute write queries, so that datasets can be updated.

2. *How can we add reasoning capabilities to the solution?*

   A simple SPARQL engine does not possess reasoning capabilities. The query results may or may not possess inferred values, depending on the underlying storage solution. When working with ontologies, a system should have reasoning capabilities to take full advantage of the semantics preserved within the data.

3. *How can we minimize the information leakage and enforce access control?*

   A data outsourcing solution should support granular access control over the shared datasets. SSE solutions always leak some information about the data, we will investigate how to minimize such leakage during query execution.

1.3 Contributions

In this thesis we aim to develop a practical and secure solution for the collaborative development, distribution and usage of ontologies.

SEnTri - A Searchable Encrypted Triplestore SEnTri is a searchable encrypted triplestore that can evaluate secure SPARQL read and write queries, possesses reasoning capabilities and supports role based access control.

P1, P2 - Two secure SPARQL evaluation protocols We developed two secure SSE protocols for the evaluation of SPARQL queries, in a dynamic and multi-user setting. The main distinction between the two protocols is in the equality function used in join operations. The first protocol relies on deterministic encryption whereas the second protocol uses additive homomorphic encryption.

A secure equality join over non-deterministic values For the second protocol we developed a novel equality join, which operates over non-deterministic values, which we denominate Eq_{Tags}. These values are encrypted using the DGK cryptosystem, an additive homomorphic scheme developed for secure integer comparison.

The prototype will be available as open-source code in a public repository, alongside the benchmark results.

1.4 Document Organization

The remaining of the document will be organized as follows:

Chapter 2 discusses related work on *Semantic Web* and *Secure Computations* and compares existing solution for secure semantic data outsourcing and SPARQL evaluation.

Chapter 3 presents the solution.

Chapter 4 describes the implementation details of each of the system's components.

Chapter 5 presents the experimental evaluation procedures, describes and discusses the obtained results.

Chapter 6 summarizes main conclusions from the research and suggests future work.

2

Related Work

In this chapter, we will discuss the architecture and design of the *Semantic Web* as well as analyzing work already done in the fields of *Secure Computations*. The remaining of the chapter is organized as follows: on Section 2.1 we discuss the *Semantic Web* stack, languages and tools; Section 2.2 describes work on *Secure Computation*, with particular attention regarding HE and SE; finally, on Section 2.3 we summarize and discuss which work is relevant.

2.1 Semantic Web: A layered architecture

In Section 1.1, we have contextualized the main philosophy and design principles of the *Semantic Web*:

1. Usage of *labeled graphs* as the data model where concepts are represented as triples consisting of a subject, predicate and object (with nodes being objects and relations between such objects modelled as the edges);

2. All data items are identified by Internationalized Resource Identifier (IRI) [24];

3. Usage of ontologies to model semantics within a domain.

Additionally, the *Semantic Web* can also be seen as a stack of various layers [5] (figure 2.1). Most of these layers are standardized due to the collective work of the World Wide Web Consortium (W3C).

Briefly describing this layered architecture, at the root level, we have the IRIs and a character set providing the baseline of communication, with distinguishable identification of resources and an agreed encoding.

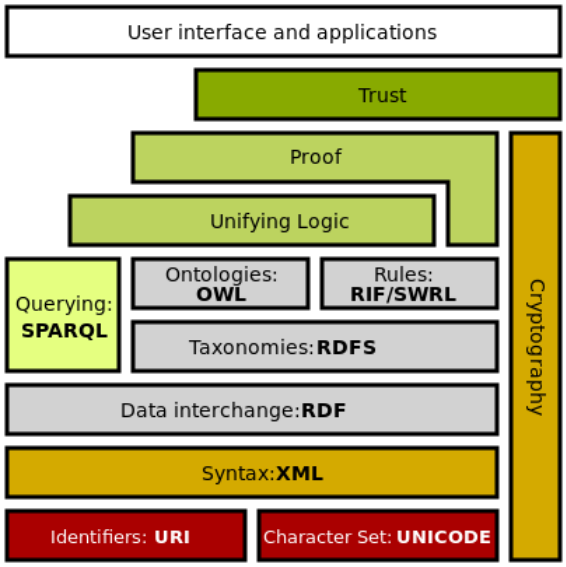

Figure 2.1: The *Semantic Web* stack [114].

At the syntax level we find the Extensible Markup Language (XML) [93]. XML supports the definition of unlimited *tags*, that can be placed in the text of a document. These *tags* facilitate the creation of hierarchy and give *structure* to the information.

From structured data we then define the basic data model using RDF [102]. With a XML like syntax, we can write statements about resources, thus embedding *semantics* within the information, as *metadata*.

The RDFS [106] is a vocabulary description language to organize RDF resources into hierarchies, based on primitives such as *classes* and *properties*. It can be seen as a rudimentary *ontology language* [5].

Ontologies are defined with OWL2 [95]. Like RDFS, it is also a vocabulary description language, but provides more expressiveness (e.g. relations between classes, *cardinality*, *equality*, richer properties with characteristics).

RDF triples can be interpreted as logical binary predicates, making reasoning possible over RDF-based data. This allows us to infer and validate ontologies, and also provide explanations on operations' results. The rules layer is a reference to this specific part of the architecture stack. There exist various languages, such as the Semantic Web Rule Language (SWRL) [113], that allow us to express rules as well as logic.

To query and retrieve any data based on RDF we use SPARQL.

The Proof and Unifying Logic layers include the notion of interconnectivity of terminologies between *knowledge bases*, with explainable proofs to operations.

Only with secure cryptographic schemes can we reach the trust layer. The security layer is still not standardized.

The top layer, references all the applications and interfaces that use this new paradigm of data representation.

In the following of this Section we will describe, in more detail, some languages and tools. We will focus on RDF in Section 2.1.1; on RDFS, in Section 2.1.2; on OWL2, in Section 2.1.3; on SPARQL, in Section 2.1.4; on common tools and software, in Section 2.1.5.

2.1.1 RDF: Resource Description Framework

2.1.1.1 Overview

RDF [102] is a flexible *domain-independent* data model. The RDF basic primitives are *resources*, *properties* and *literals*. We can construct *statements* about resources by organizing these basic primitives in sets of *triples*. A triple is composed of a *subject*, a *predicate* and an *object*. In a statement (triple) about a subject (always a resource) we define the value (object) of a property (predicate), which can be another resource or a literal.

Literals are atomic values (e.g. strings, numbers). Resources and properties, respectively, model the "things"and relationships between such "things". Both are identified using IRIs. Resources without an IRI are called *blank nodes* (or *bnodes*).

To make a statement B about a statement A we can use a technique called *reification*. Reification consists in: firstly, defining a statement B where the object $objA$ represents the statement A; secondly, creating the properties *subject*, *predicate* and *object* mapped from $objA$ to the original subject, predicate (now a resource), and original object of statement A [5].

2.1.1.2 Syntax

Graphical RDF being a data model, can be written using different syntaxes. Graphically we can represent statements as a labeled and directed graph. The *nodes* are labelled with the resources IRIs, a literal value or left blank, whereas *arcs* are labelled with the properties IRIs. The arcs are directed from subject to object of a statement. Figure 2.2 provides an example of a RDF graph.

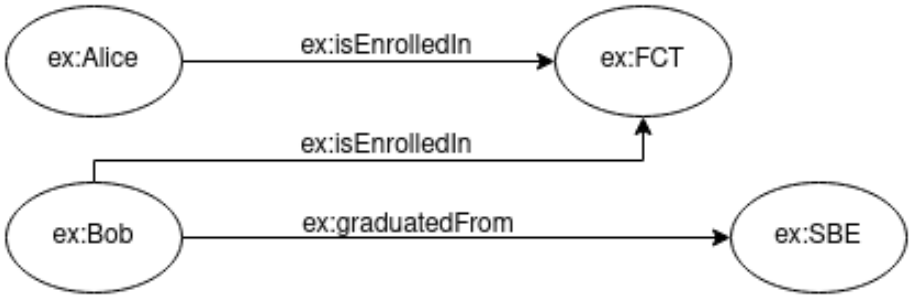

Figure 2.2: A simple RDF graphical example.

Text-Based There are multiple text-based syntaxes for RDF. Some of the most commonly used are TURTLE [104] (listing 2.1), and its extension for named graphs *TriG* [103]. In TURTLE, IRIs are enclosed by angle brackets. The subject, property, and object are written in this order. Statements are finished by a period. Literals are surrounded in quotes with the type of data specified with an IRI. The data value and type are separated by ^^ (if no data type is mentioned, the value is inferred as being a string). We can also define IRI abbreviations (using @prefix, followed by the prefix to substitute the IRI, a colon, the IRI and a period) and simplify statements, for easier readability (e.g.using a semicolon to make more than one statement about the same subject, defining statements with nested blank nodes using square brackets). Comments start with #.

```
# FCT is located in Almada
<http://www.my-custom-domain/faculties.ttl#FCT> <http://www.my-custom-domain/ontology/
    ↪ location> <http://www.my-custom-domain/resource/Almada> .

# FCT has 8.000 students
<http://www.my-custom-domain/faculties.ttl#FCT> <http://www.my-custom-domain/faculties.
    ↪ ttl#asNumberOfStudents> "8000"^^<http://www.w3.org/2001/XMLSchema#integer> .
```

Listing 2.1: Simple RDF example, written in TURTLE syntax.

XML/RDF [105] is a standardized XML-based syntax and *RDFa* [107] permits RDF to be embedded in the HyperText Markup Language (HTML)[1].

2.1.2 RDFS: RDF Schema

2.1.2.1 Overview

RDF is a flexible data-model. However, for interconnectivity of different RDF knowledge bases, semantics for restricting meaning in a specific domain are needed.

RDFS [106] does that by providing vocabulary to describe groups of related resources and the relationships between these resources. This vocabulary is a set of RDF resources identified under the `http://www.w3.org/2000/01/rdf-schema#`.

2.1.2.2 RDFS primitives

Classes group resources together, the *instances* of the class. *Properties* are defined by specifying to which classes of resource they may apply, using the mechanism of *domain* and *range*. This allows for easy extension, by not having to redefine classes wherever a new property, that applies to an existing class, is needed.

To state that a resource is an instance of a class we use the *rdf:type* property.

The *rdfs:subClassOf* is used to define that all the instances of a class are instances of its *superclasses*: if *A* rdfs:subClassOf *B*, then instances of *A* are also instances of *B*. The *rdfs:subPropertyOf* states that instances related by a property are also related by its *superproperties*: if *A* rdfs:subPropertyOf *B*, then instances related to *A* are also related to *B*.The properties rdfs:subClassOf and rdfs:subPropertyOf are *transitive*, and there are no restrictions on the number of superclasses or superproperties that, respectively, a class or a property can have.

The *rdfs:domain* property defines that any resource with a given property is an instance of the domain classes that the property applies to. The *rdfs:range* property specifies that the values of the property are instances its range classes.

In tables 2.1 and 2.2, we summarize the core RDFS classes and properties. In figure 2.3, we extend the RDF graphical example 2.2 with RDFS.

[1] We recommend the curious reader to read the W3C recommendations to better learn the syntaxes.

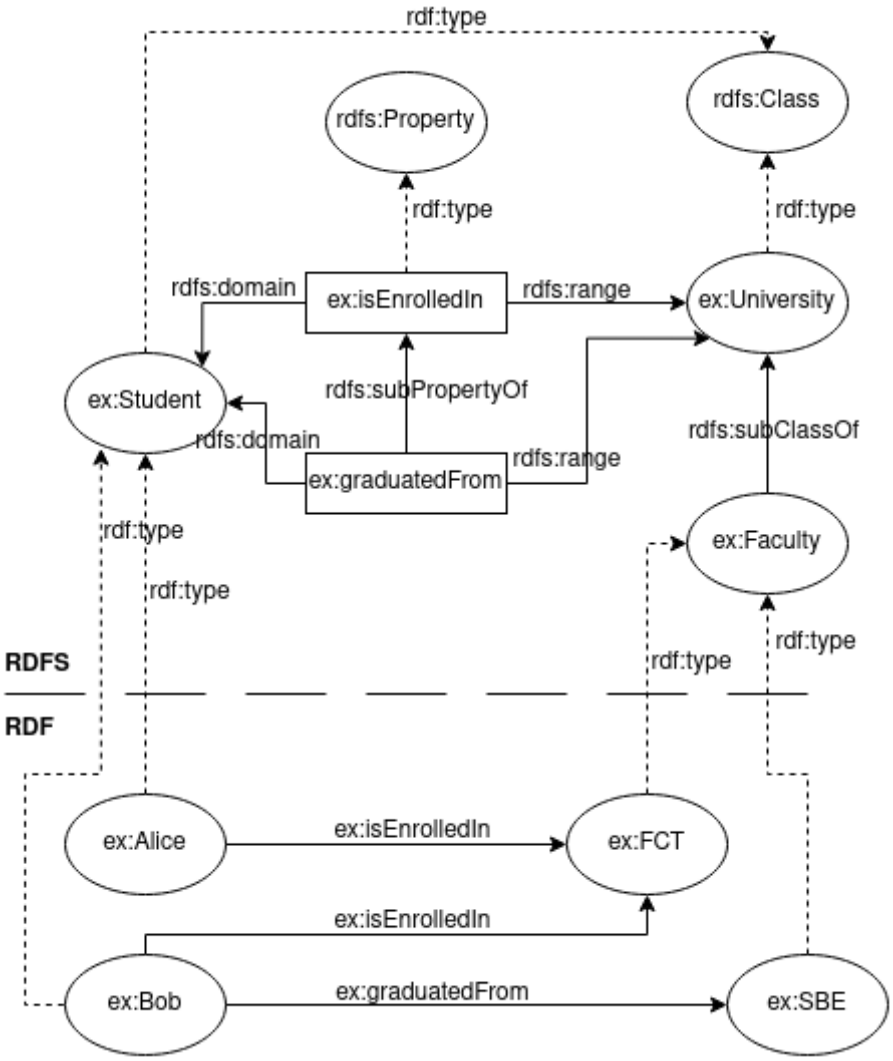

Figure 2.3: A simple RDFS graphical example.

RDFS Core Classes	
Class	**Description**
rdfs:Resource	The class resource, everything.
rdfs:Class	The class of classes.
rdfs:Literal	The class of literal values, e.g. textual strings and integers.
rdf:Property	The class of RDF properties.
rdfs:Statement	The class of RDF statements.

Table 2.1: RDFS Core Classes and respective description (rdfs:comment).

Core RDFS Properties			
Property	**Description**	**Domain**	**Range**
Relationship Definition			
rdf:type	The subject is an instance of a class.	rdfs:Resource	rdfs:Class
rdfs:subClassOf	The subject is a subclass of a class.	rdfs:Class	rdfs:Class
rdfs:subPropertyOf	The subject is a subproperty of a property.	rdf:Property	rdf:Property
Property Restriction			
rdfs:domain	A domain of the subject property.	rdf:Property	rdfs:Class
rdfs:range	A range of the subject property.	rdf:Property	rdfs:Class
Reification			
rdf:subject	The subject of the subject RDF statement.	rdf:Statement	rdfs:Resource
rdf:predicate	The predicate of RDF statements.	rdf:Statement	rdfs:Resource
rdf:object	The object of RDF statements.	rdf:Statement	rdfs:Resource
Utility			
rdfs:seeAlso	Further information about the subject resource.	rdfs:Resource	rdfs:Resource
rdfs:isDefinedBy	The definition of the subject resource.	rdf:Resource	rdfs:Resource
rdfs:comment	A description of the subject resource.	rdf:Resource	rdfs:Literal
rdfs:label	A human-readable name for the subject.	rdf:Resource	rdfs:Literal

Table 2.2: RDFS Core Properties and respective description (rdfs:comment).

2.1.3 OWL2: Web Ontology Language

2.1.3.1 Overview

An *ontology* was defined by Studer et al. as "a formal, explicit specification of a shared conceptualisation"[83]. An ontology should specify consensual and machine-readable knowledge by means of a formal definition of concepts, where concepts have explicit types and restrictions. Therefore, an *ontology language* should provide a well-defined syntax, formal semantics, convenient and sufficient expressiveness and efficient reasoning support.

RDFS can be considered a very limited *ontology language*. OWL2 [95] provides a number of additional features (e.g. cardinality constraints, class equivalence, intersection and disjunction). OWL2 does not simply extend RDFS, because RDFS primitives are too expressive. This would make reasoning undecidable [5]. As such, OWL2 is divided in two sublanguages: *OWL2 Full* [99], a direct extension of RDFS with all OWL2 semantics; *OWL2 DL* [94], a language with some restrictions on the way OWL2, RDF and RDFS primitives may be used.

The correspondence theorem of [94] states that: given an *OWL2 DL* ontology, inferences drawn using its *Direct Semantics* will still be valid if the ontology is mapped into an RDF graph and interpreted using the *RDF-Based Semantics*, the semantics of *OWL2 Full*.

OWL2 can be expressed with any valid RDF syntax, however there are various specific syntaxes: the Functional-Style Syntax [100], which is very compact and was the language used in the formal specification of the language; the Manchester Syntax [96], designed to be human-readable (it is very popular in user interfaces of semantic editors); and the OWL/XML Syntax [101], which allows easier processing with XML tools.

Figure 2.4 details the structure of OWL2. Finally, *OWL2* can have different specification, called profiles [97], which provide reasoning in polynomial time. The coverage of OWL2 profiles is not on the scope of this work.

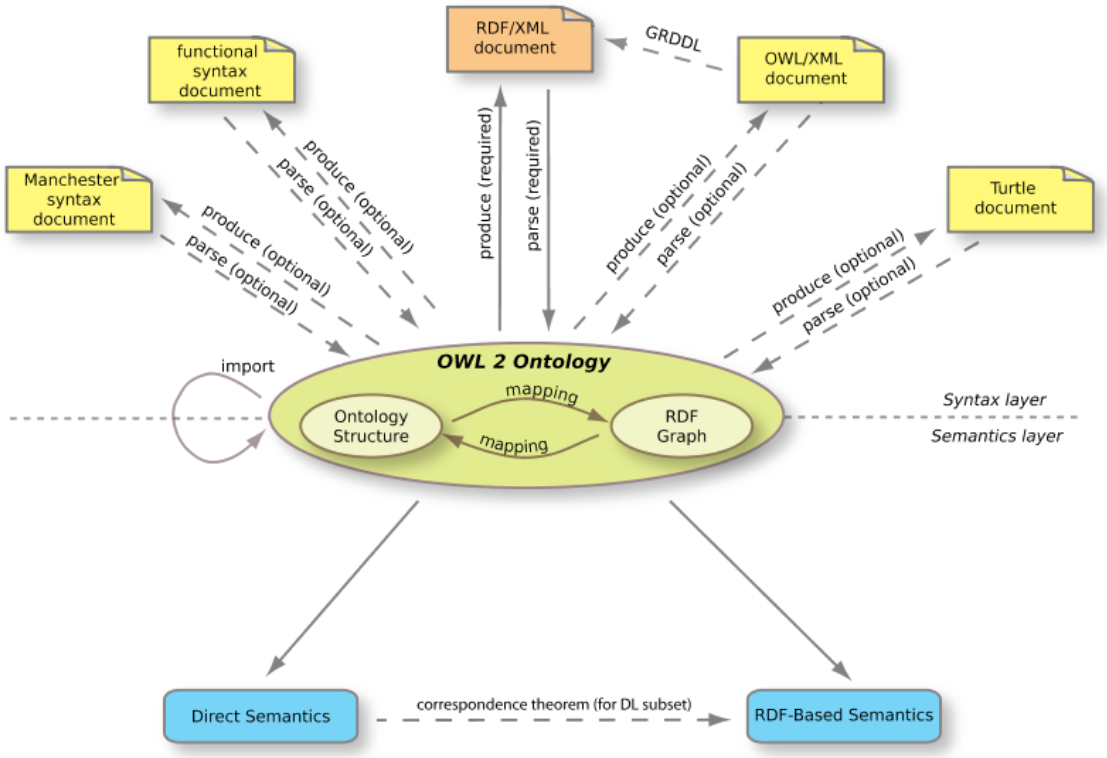

Figure 2.4: The OWL2 Structure [95].

2.1.3.2 OWL Language

In OWL2 the term individual is equivalent to instance in RDFS. An *assertion* is a statement about a resource. *Expressions* are a combination of classes, properties and instances. An *axiom* relates such expressions to classes[2]. Listing 2.2 provides a simple OWL2 example.

Individual Assertions

In OWL2 *assertions* about individual resources can be used to define their type, class membership, properties and even provide ways to define identity and negative statements. More specifically on the identity statements, because OWL2 has an open world assumption, we can never differentiate two individuals just because they have different IRIs. To do so we use owl:differentFrom. To explicitly state negative facts we use owl:NegativePropertyAssertion [5].

[2]For an overview of the language we recommend the reader to consult the reference guide [98].

```
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

# The base namespace of the FCT ontology
@base<http://www.my-custom-domain/fct.ttl> .

<http://www.my-custom-domain/fct.ttl>
    rdf:typeowl:Ontology ;
    rdfs:label"FCT Ontology"^^xsd:string ;
    rdfs:comment "An example OWL2 ontology"^^xsd:string ;
    owl:versionIRI <http://www.my-custom-domain/fct.ttl#1.0> .

# Class Expression
_:x rdf:type owl:Class ;
    owl:unionOf ( :Mandatory :Optional ) .

# Class Axiom
:Course owl:equivalentClass _:x .

# Class Assertion
:class101 rdf:type :Course .
```

Listing 2.2: Simple OWL2 example, written in TURTLE syntax.

Classes

In OWL2 every resource that is of type owl:Class is a class. The two classes defined by default are *owl:Thing* and *owl:Nothing*. Respectively, these two specify the superclass of all classes and the subclass of all classes: every individual is a member of owl:Thing and no individual can be member of owl:Nothing. This is very useful for reasoning, because inconsistency is detected when classes are equivalent to owl:Nothing (have no individuals in their membership).

Class Axioms In OWL2 we can relate *expressions* to classes using *axioms*. These can define:

- *Subclass Relations*, with equivalent usage like in RDFS;
- *Class Equivalence*, in equivalent classes every individual must be a member of each class;
- *Enumerations*, a type of class definition via explicitly stating all individuals who belong to the class membership;
- *Intersection*, in a class described as the intersection of various classes, all of its members must be also individuals belonging to the membership of each of those classes;
- *Union and Disjoint Union*, in a class described as the union of various classes, every individual belonging to its membership must also belong to the membership of at least one of those classes. In the particular case of disjoint union, the intersection of the classes is equivalent to owl:Nothing;

- *Disjoint Classes*, in disjoint classes no individual can be a member of both class (the intersection of the classes is equivalent to owl:Nothing);

- *Complement*, complement classes contain all the individual such that the union of both classes is equivalent to owl:Thing.

Properties

OWL2 has three types of properties: *object* properties, which relate individuals to other individuals; *datatype* properties, which relate individuals to literals; and *annotation* properties; which are used to specify human-readable labels, comments and explanations.

The *top* and *bottom* properties specify, respectively, properties that relate to all and no individuals: owl:topObjectProperty is the superproperty of all owl:ObjectProperties and owl:bottomObjectProperty relates to no individual; owl:topDataProperty is the superproperty of all owl:DatatypeProperties, relating an individual to any possible literal value, and owl:bottomDataProperty relates no individual to any literal value.

Properties can have different characteristics:

- *Transitive Properties*, if a relates to b via property p and c relates to a via property p, then c relates to b via property p;

- *Symmetric Properties*, if a relates to b via property p, then inverse is true;

- *Asymmetric Properties*, if a relates to b via property p, then the inverse is $\perp$;

- *Functional Properties*, if for each instance the property can only have one unique value;

- *Inverse-functional Properties* if the object of a property statement relates to a single subject;

- *Reflexive Properties*, if every individual relates to itself via the property;

- *Irreflexive Properties*, if no individual relates to itself via the property.

Property Axioms Just like class axioms, we can specify *property axioms* to describe the way properties relate to classes and other properties. These allow us to define:

- *Domain and Range*, used to determine class membership of individuals;

- *Inverse Relationship*, used to specify the inverse of the property;

- *Equivalence*, used to specify equivalent properties. Individuals related by one property will always relate by all equivalent properties;

- *Disjointness*, used to specify disjoint properties, where individuals related by one property can never be related by the other;

- *Property Chains*, used to aggregate connected properties under another property.

More Granular Class Axioms

We can write more granular class definitions in OWL2 than by simply defining class axioms. This is achieved by combining class and property axioms. It is done by defining a class axiom that relates to an anonymous class (owl:Restriction) which contains axioms on its properties. The anonymous class definition captures all individuals that satisfy such restrictions. As such we can define:

- *Universal and Existential Restrictions* which the members of the class must satisfy. The class is defined as a subclass of an anonymous class with property restrictions on, respectively, owl:allValuesFrom and owl:someValuesFrom;
- *Necessary and Sufficient Conditions*, relating a class to a property, using rdfs:subClassOf, does not provide sufficient information to conclude whether an individual belongs to the class membership or not (it only states necessary, not sufficient, conditions for class membership). With owl:equivalentClass, it becomes clear that an individual belongs to the class membership, if it satisfies the restriction (the conditions are necessary and sufficient).
- *Value*, *Cardinality*, *Data Range and Datatype Restrictions* on the anonymous class, limiting the value, cardinality of the value, range of possible values and the datatype of the property value relating to members of the superclass;
- *Self Restrictions* on the individuals captured by the anonymous class.

2.1.3.3 OWL2 DL

Contrary to *OWL2 Full*, *OWL2 DL*, being less expressive, retains *decidability*. In this Section, we specify the restrictions imposed by *OWL2 DL* [5]:

- The application of OWL2 primitives to each other is not allowed; only classes and non-literals resources may be defined, with all classes being individuals of owl:Class;
- Properties are disjoint individuals of either owl:ObjectProperty or owl:DatatypeProperty;
- Annotation properties are ignored by reasoners;
- Properties, for which range includes non-literal resources, are separated from those which relate to literal values;
- A resource can only be a class, property, or instance at the same time. Classes, properties or instances with the equal names will be treated as distinct (a mechanism called punning);
- Only functional properties can be assigned to datatype properties;
- Only object properties can have an inverse;
- Property chains can only involve object properties;
- Self restrictions on datatype properties are not allowed.

2.1.4 SPARQL

2.1.4.1 Overview

In the previous Section we've discussed the structure of OWL2. OWL2 documents have a direct mapping to RDF documents. As such, we can publish and query OWL2 ontologies using the standard approach for RDF data.

RDF triples are stored in *triple stores*, specialized databases for this type of data. The underlying architecture of such databases varies, from relational to NoSQL approaches [21, 54]. The recommended query language to use is SPARQL [110]. It provides a range of operations over published RDF graphs, accessible via SPARQL endpoints: from read queries [112] to write operations [111, 109].

2.1.4.2 SPARQL Queries

In this section we will mathematically define core components of *SPARQL*, briefly discuss the basis of functioning for this type of queries, explain how they are evaluated, given the semantics of *graph patterns*, and cover their syntax.

Defining an RDF dataset Let I, B, L be disjoint sets of IRIs, blank nodes, and literals, respectively. RDF data is a set of subject, predicate and object triples $t = (s, p, o)$, where $s \in I \cup B$, $p \in I$ and $o \in I \cup B \cup L$.

Let V be a set of nodes formed by subjects and objects and E be a set of edges, labeled by the corresponding predicate. A set of RDF triples can be represented as a directed, edge-labeled, graph $G = (V, E)$. An RDF dataset is set of *named* graphs and the *default* unnamed graph. We will define such dataset as $D = \{G, \gamma\}$, where γ is a function which assigns a named graph $\gamma(i)$ to each i in a finite set $dom(\gamma) \in I \cup B$ [69, 74].

Defining a SPARQL Query Let T = {SELECT, ASK, CONSTRUCT, DESCRIBE}, a SPARQL query can be defined as a tuple $q = (t_q, d, m, P_q)$ where $t_q \in T$ is the *query type*, $d \in I \cup B$ is an optional term identifying the RDF graph to be queried, m is a set of *solution modifiers* and P_q is a *graph pattern*.

Defining a graph pattern Let $V = \{?v_i, ?v_{i+1}, ..., ?v_{i+n}\}$, where $n \in \mathbb{Z}^+$, be a set of variables, disjoint from I, B, and L. We will denote the set of variables in P as $vars(P)$. A triple pattern is a tuple $tp = (s', p', o')$ such that $s' \in I \cup V$, $p' \in I \cup V$ and $o' \in I \cup L \cup V$. A set of triple patterns is usually referred as a Basic Graph Pattern (BGP). A *property path pattern* pp can be inductively defined as an element of $(I \cup B \cup V) \times pp \times (I \cup B \cup L \cup V)$. Finally, a *graph pattern* is an expression specified by the following grammar:

$$\begin{aligned} P ::= \; & tp \mid pp \mid q \mid P_1 \,.\, P_2 \mid P \text{ FILTER } R \mid P_1 \text{ UNION } P_2 \mid P_1 \text{ OPTIONAL } P_2 \mid \\ & P_1 \text{ MINUS } P_2 \mid \text{Graph } i \; P \mid \text{Graph } v? \; P \end{aligned} \tag{2.1}$$

Here tp is a triple pattern, pp is a *property path pattern*, q is a subquery, $i \in I$, $?v \in V$ and R is a SPARQL filter constraint[3]. To remove ambiguities parentheses can be added to a pattern. Finally, we'll define the set of variables that occur in a pattern P as $vars(P)$ [69].

[3] For a detailed explanation of the syntax and semantics, we recommend the reader to consult section 17 of [112]

How do SPARQL queries work? An isomorphism of two graphs is an adjacency preserving bijection between their vertex sets [52].

The basis of SPARQL queries is finding matches with *graph patterns* in the RDF dataset [47, 21]. This problem is equivalent to the *subgraph isomorphism problem*, a generalization of the graph isomorphism problem: given two graphs G_1 and G_2, we try to find if G_1 contains a subgraph that is isomorphic to G_2.

In the context of a SPARQL query q, G_1 is a published RDF graph identified by d and G_2 corresponds to a *graph pattern* P_q. From this match we obtain a set of *bindings*: a multiset of mappings between $vars(P)$ and data within G_d. The final result of a query will also depend on t_q, the *query type*, and m, the specified *solution modifiers* (i.e. aggregation, grouping, sorting, duplicate removal).

Semantics of SPARQL graph patterns A *mapping* μ is a partial function $\mu : V \to I \cup B \cup L$ that assigns values to a finite set of variables. The domain of μ, denoted by $dom(\mu)$, is the subset of V where μ is defined.

Two *mappings* μ_1 and μ_2 are compatible, denoted by $\mu_1 \sim \mu_2$, if $\mu_1(?v) = \mu_2(?v)$ for all common variables $?v \in dom(\mu_1) \cap dom(\mu_2)$.

Abusing notation, we define $t = \mu(tp)$ as the triple obtained after replacing the variables in the *triple pattern* tp according to μ [48, 69]. Finally, let $\mu \vDash R$ represent a mapping that satisfies a filter condition R.

We define the *join*, *union*, *difference*, and *left outer join* operations on two sets of mappings Ω_1 and Ω_2 as follows:

$$\begin{aligned}
\Omega_1 \bowtie \Omega_2 &= \{\mu_1 \cup \mu_2 \mid \mu_1 \in \Omega_1,\ \mu_2 \in \Omega_2,\ \mu_1 \sim \mu_2\} \\
\Omega_1 \cup \Omega_2 &= \{\mu \mid \mu \in \Omega_1 \vee \mu \in \Omega_2\} \\
\Omega_1 - \Omega_2 &= \{\mu_1 \in \Omega_1 \mid \forall \mu_2 \in \Omega_2,\ \mu_1 \nsim \mu_2\} \\
\Omega_1 \leftouterjoin \Omega_2 &= (\Omega_1 \bowtie \Omega_2) \cup (\Omega_1 - \Omega_2)
\end{aligned} \tag{2.2}$$

The semantic evaluation of P over a dataset D is a set of mappings $[\![P]\!]_D$. Inductively, we define it as:

$$\begin{aligned}
&[\![tp]\!]_D = \{\mu \mid dom(\mu) = vars(tp) \wedge \mu(tp) \in G_D\} \\
&[\![pp]\!]_D = \{\mu \mid dom(\mu) = vars(pp) \wedge \mu(pp) \in G_D\} \\
&[\![q]\!]_D = \{\mu \mid dom(\mu) = vars(P_q) \wedge \mu(P_q) \in G_D\} \\
&[\![P_1 \,.\, P_2]\!]_D = [\![P_1]\!]_D \bowtie [\![P_2]\!]_D \\
&[\![P_1 \text{ UNION } P_2]\!]_D = [\![P_1]\!]_D \cup [\![P_2]\!]_D \\
&[\![P_1 \text{ OPTIONAL } P_2]\!]_D = [\![P_1]\!]_D \leftouterjoin [\![P_2]\!]_D \\
&[\![P_1 \text{ MINUS } P_2]\!]_D = [\![P_1]\!]_D - [\![P_2]\!]_D \\
&[\![P_1 \text{ FILTER } R]\!]_D = \{\mu \in [\![P]\!]_D \mid \mu \vDash R\} \\
&[\![\text{Graph } i\ P]\!]_D = \{\mu \in [\![P]\!]_{(\gamma(i),\gamma)} \mid i \in dom(\gamma)\} \\
&[\![\text{Graph } ?v\ P]\!]_D = \bigcup_{i \in dom(\gamma)} \{?v \to i\} \bowtie [\![P]\!]_{(\gamma(i),\gamma)}
\end{aligned} \tag{2.3}$$

Syntax *SPARQL* queries have a similar structure to the Structured Query Language (SQL) queries: we define the operation to execute using a keyword, followed by the

projection of variables to be shown in the result, the keyword *WHERE*, the *graph pattern*, enclosed in brackets, and, optionally, some modifiers of the result (i.e. *ORDER BY* to sort results and *DISTINCT* to remove duplicates). Additionally, we may also specify the RDF graph to consider for the query evaluation with the keywords *FROM* or *FROM NAMED*. Furthermore, the SPARQL syntax is very similar to TURTLE, so we can specify prefixes for namespaces. The basic *SPARQL 1.1 Query Language* [112] provides the following query types:

- *SELECT*: used to extract values from a RDF graph. The result is a set of bindings;
- *ASK*: returns a boolean, true if there was a match between the *graph pattern* and the RDF graph, false otherwise;
- *CONSTRUCT*: we specify two *graph patterns*, one to fetch *bindings* and another to use as a template for constructing new triples, from the values in the *bindings*. The final result is the set of newly generated triples.
- *DESCRIBE*: results depend on implementation [69], but usually returns a set of RDF triples describing the matching patterns.

In the following listings 2.3, 2.4 and 2.5 we show a few query examples.

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix fct: <http://www.my-custom-domain/fct.ttl>

# Operation and variables
SELECT ?opt_course ?man_course ?popular_course
# Query Pattern
WHERE
{
    ?opt_course rdf:type fct:Optional .
    ?man_course rdf:type fct:Mandatory
}
```

Listing 2.3: SPARQL query with a conjuntion pattern

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix fct: <http://www.my-custom-domain/fct.ttl>

# Operation and variables
SELECT DISTINCT ?popular_course
# Query Pattern
WHERE
{
    ?popular_course fct:hasStudentEnrolled ?num_students .
    FILTER (?num_students > 60)
}
# Optional Modifiers
LIMIT 20
```

Listing 2.4: SPARQL query with filters and modifiers

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix fct: <http://www.my-custom-domain/fct.ttl>
@prefix nova: <http://www.my-custom-domain/nova.ttl>

# Operation and variables
SELECT DISTINCT ?optional_course
# Query Pattern
WHERE
{
    {?optional_course rdf:type fct:Optional}
    UNION
    {?optional_course rdf:type nova:Elective}
}
```

Listing 2.5: SPARQL query with a disjunction pattern

Besides read operations, *SPARQL Update* [111], provides additional options for inserting, publishing and deleting data in triplestores:

- *INSERT/DELETE*: these operations add and delete, respectively, newly generated triples, based on a template, into a destination graph (if it does not exist it should be created). Like in a *CONSTRUCT* query we specify two *graph patterns*, one to fetch bindings and another to use as a template for constructing triples. We can join both operations, in this situation we specify three *graph patterns*, two to use as templates to generate the triples to be inserted or deleted and a third one to fetch bindings.

- *INSERT/DELETE DATA*: similar to their pattern-based variants, but in these operations we specify concrete data inline.

- *LOAD*: used to read the contents of a document that represents a graph and saves it into a destination graph in a triplestore.

- *CLEAR*: this operation removes all the triples in the specified graphs.

2.1.5 Tools & Development Software for Ontologies

To developed and maintain ontologies, professionals use advanced semantic editors. *Protégé* [56] is a free, open-source, and the usual choice. It provides a rich ontology editing environment with full support for OWL2, available in a desktop version or on the web [90]. Many of its functionalities are based on plugins: automatic verification of errors and consistency in the design using reasoners like *Pellet* [80]; debugging tools; query answering; collaborative development [91]; graphic visualization of data and knowledge.

To store RDF data, popular storage solutions are: *MarkLogic* [50], *Virtuoso* [62], *Amazon Neptune* [4], *GraphDB* [61], *Apache Jena - TDB* [88]. While *Apache Jena - TDB* is primarily a RDF store, the rest are *multi-model* database systems. To test their capability to handle SPARQL queries, the Berlin SPARQL Benchmark (BSBM) is a standard benchmark [17]. The Lehigh University Benchmark (LUBM) is another benchmark, which is tailored for OWL2 data [36]. It provided synthetic data generation for a single realistic ontology.

Finally, for developers, many Application Programming Interface (API)s [63] or full framework solutions, such as *Apache Jena* [87], are available for building *Semantic Web* applications.

2.2 Secure Computations

In Section 1.1, we have contextualized the motivations behind the need for secure computation when sharing ontologies over cloud or other distributed networks.

In this setting we consider the *honest-but-curious*, or *passive*, adversary model. As such an adversary will follow the protocol properly but keep a record of all intermediate computations [34] in an attempt to learn private information while observing views of the protocol execution [26]. Security concerns should take into account the operation history, data stored in the server and also the memory access pattern, as the adversary will have access to all of this information.

The *Encryption at Rest* approach encrypts all data stored in the untrusted server. To perform computations we need to download the full dataset, decrypt it, and only then can we execute operations on the data. This is too inflexible and computationally expensive.

ORAM [35] fits well this adversary model, as it allows for the concealment of memory access patterns by shuffling and continuously re-encrypting data on memory, but it is still impractical.

When used exclusively, Homomorphic Encryption [3, 31] schemes are not efficient enough to be feasible in real-world applications [11]. However, they are applied in conjunction with other security schemes, such as in some Searchable Encryption schemes [11, 16]. For this reason, in the following Sections we will discuss related work in the fields of HE, in Section 2.2.1, and SE, in Section 2.2.3.

2.2.1 Homomorphic Encryption

The term homomorphism, in security, was used for the first time by Rivest et al. in 1978 [75]. Homomorphic Encryption is a cryptographic method based on the concept of algebra homomorphism, where both decryption and encryption are seen as homomorphisms of the relations between the plaintext and the ciphertext [3, 31]. With HE, computations done over encrypted data, are preserved after decryption of the ciphertext.

From [3], we define an encryption scheme to be homomorphic over an operation $\bullet$ if it satisfies the following equation, where E is the encryption algorithm and M the set of all possible messages:

$$E(m_1) \bullet E(m_2) = E(m_1 \bullet m_2), \forall m_1, m_2 \in M \tag{2.4}$$

Some HE schemes preserve additive properties [64], others maintain multiplicative properties [76, 25], and some conserve both additive and multiplicative properties [33]. The HE schemes that preserve both properties allow for any arbitrary operation, because boolean circuits can be represented using only *XOR* (addition) and *AND* (multiplication) gates [3].

HE is classified in four main types:

- Fully Homomorphic Encryption, which supports any number of operations with no restriction on the number of operations;
- Somewhat Homomorphic Encryption, which permits some types of operations, a limited number of times;
- Partially Homomorphic Encryption, that allows for one type of operation with no restriction on the number of times;

- Leveled-Fully Homomorphic Encryption [12], which supports any type of operations for a limited, and expected, number of L times (referred to as depth-L).

2.2.2 Property-preserving Encryption

Homomorphic encryption allows arbitrary computations to be executed on encrypted data while not leaking any information about the original plaintext. For operations such as equality joins, where we require equality checks, we do need to preserve (leak) certain properties from the original plaintexts. This type of encryption is known as property-preserving encryption.

Deterministic Encryption Deterministic encryption is type of property-preserving encryption scheme. Contrary to a probabilistic scheme, it will always generate the same ciphertext for each plaintext. This allows equality checks to be performed on the encrypted data. Naturally, its security guaranties are weakened. Deterministic ciphertexts are vulnerable to statistical analysis attacks.

Order-Preserving Encryption (OPE) These type of encryption schemes preserve the order guaranties of the plaintext data. Naturally, the order relations between the ciphertext are leaked. Some implementations can leak as much as half of the data bits of the ciphertexts [72].

2.2.3 Searchable Encryption

2.2.3.1 Overview

Searchable Encryption uses classical encryption schemes to perform encrypted search queries. While the first approach implemented it via a searchable ciphertext [81], most modern schemes do it using encrypted searchable indexes. Searchable Encryption can be divided in SSE, when it is based on symmetric key cryptography, and PEKS, when public key algorithms are used. We will focus on index-based SE schemes, more specifically in index-based SSE ones, due to their higher efficiency compared to PEKS approaches.

2.2.3.2 SE General Index-Based Model

A general model, for the index-based SE schemes was defined in [11]. Given a database $DB = (M_1, ..., M_n)$ consisting in a set of n plaintext documents M, some data items (e.g, keywords in the documents) $W = (w_1, ..., w_m)$ are extracted from the documents and encrypted using a key K, using an encryption algorithm Enc. The documents may be encrypted too, generating a collection of ciphertexts C, usually with a different key K'. A client generates an encrypted index I, for the database DB, using a $BuildIndex$ function. The client stores the following information in an *honest-but-curious* untrusted server:

$$\begin{aligned} I &= BuildIndex_K(DB = (M_1, ..., M_n), W = (w_1, ..., w_m)) \\ C &= Enc_{K'}(DB) \end{aligned} \tag{2.5}$$

To perform search queries, the client generates a trapdoor $T = Trapdoor_K(f(w_j))$, for a predicate f a keyword w_j. With trapdoor T, the server performs the search over I, using a $Search$ algorithm, by checking for an encrypted index $i = Search(I, T)$. If there is a keyword w that satisfies the predicate f, it returns to the client the corresponding

ciphertexts associated with i or $\perp$ if no match was found. In figure 2.5 we demonstrate the interaction between client and server.

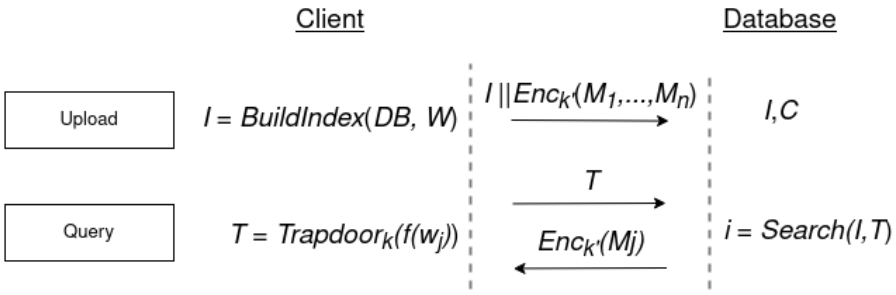

Figure 2.5: Interaction between client and server.

2.2.3.3 Client/Server Model

In SE schemes, one or various clients, the *writers* store encrypted data in a server. To issue a query request to the server, one or multiple clients, the *readers*, generate trapdoors. There are four possible SE scheme architectures:

- Single Writer/Single Reader (S/S);
- Single Writer/Multiple Reader (S/M);
- Multiple Writer/Single Reader (M/S);
- Multiple Writer/Multiple Reader (M/M).

The S/S architecture is usually used for *data outsourcing* and M/S, S/M and M/M employed in *data sharing* scenarios [11]. Symmetric key schemes are typically S/S, while public key schemes M/S (multiple users can write using a *public key*, but only the owner of the *private* key can generate trapdoors). Extension of the single reader schemes to multiple reader schemes can be done with *key sharing* or with the usage of *proxy re-encryption*.

In proxy re-encryption cryptography, a proxy is used to re-encrypt a ciphertext, encrypted using key K, and generates a new ciphertext, that is decrypted using key K', where $K \neq K'$ and the decrypted plaintext remains unchanged. Proxy re-encryption is usually employed with public key cryptography [8, 39, 23], but there are some schemes that support the symmetric key setting [86, 77, 67].

2.2.3.4 Information Leakage

Almost all SE techniques have privacy issues because they leak some information [11]. This leakage can be *index* information, the *search pattern*, or *access pattern* during execution:

- *Index Information* refers to the information about the index keywords. It may include the number of keywords per document or database, the total of documents, document similarity, their length and metadata;
- *Search Pattern* includes the information about keywords, and predicates, that is inferred by correlating equal results from different searches. The search pattern can be targeted using statistical analysis;

- *Access Pattern* includes the information that may be derived from the query results, based on the memory accesses needed to retrieve the documents. (e.g. one can infer whether a query is more restrictive than another, based on the total of documents retrieved; or if some files are more accessed than others).

2.2.4 Symmetric Searchable Encryption

2.2.4.1 Overview

SSE is a type of SSE based on symmetric key cryptography. We will first review the standard security definitions for SSE and then focus on describing a few reference schemes.

2.2.4.2 Security Definitions

Indistinguishability Against Chosen Plaintext Attacks (IND-CPA) is a property commonly used to evaluate encryption schemes, particularly in the context of public-key or symmetric-key cryptography. Informally, a scheme is IND-CPA secure if an adversary A, given two arbitrary plaintext messages and their respective ciphertexts, cannot distinguish which ciphertext corresponds to each message with a probability higher than that of random chance. This probability upholds even if A can adaptively query the encryption oracle.

This definition is not appropriate for SE schemes where the leakage of information originates from trapdoors and queries.

In 2006, Curtmola et al. [22] revisited the security definitions for SSE schemes, contributing with two new definitions, which are still the standard in use: Indistinguishability Against Non-Adaptive Chosen Keyword Attack (IND-CKA1) and Indistinguishability Against Adaptive Chosen Keyword Attack (IND-CKA2).

Both require that only the outcome of search patterns may be leaked. Additionally, they guarantee that trapdoors do not leak information about the keywords, apart from what can be inferred from the search and access pattern. Their main difference is whether the adversary can produce an adaptive or non-adaptive attack.

- IND-CKA1 guarantees security if the client generates all queries at once;
- IND-CKA2 guarantees security even if the client chooses the queries based on previously obtained trapdoors and search outcomes.

2.2.4.3 SSE Schemes

Song et al. In 2000, the first practical SSE scheme was proposed [82]. The general basis of their method is to encrypt each word of a document and embed hash values with a specific format in the ciphertext. During search, the server executes a *sequential scan* of the document, extracts the hashes and checks for matches on their values.

First, the plaintext is divided into *fixed-size* blocks w_i and each one is encrypted with a *deterministic*[4] encryption algorithm Enc, using key k. The ciphertexts $X_i = Enc_k(w_i)$ are split into two parts $X_i = \langle l_i, r_i \rangle$. Using a *stream cipher*[5] as a Pseudo-Random Function (PRF), a pseudo-random string of bytes $S_i = PRF_{k'}(i)$ is generated from the index position using key $k' \neq k$. Then, a hash $Y_i = \langle S_i, F_{k_i}(S_i) \rangle$ of this string of bytes is computed using a keyed hash function F_{k_i}, where the key is the output of a pseudo-random function

[4] Deterministic encryption leaks message equality.

[5] For information on stream ciphers we recommend reading chapter 8.4 of [66].

$k_i = PRF_{k''}(l_i)$ that takes l_i as an input, with key $k''' \neq k''$. Finally, we compute the *XOR* of X_i with S_i to generate the final ciphertexts $C_i = X_i \oplus Y_i$.

To search, we generate a trapdoor $T = \langle X, K_s \rangle$, where $X = E_k(w) = \langle L, R \rangle$ is the encryption of keyword w and $K_s = PRF_{k''}(L)$. A server searches for the keyword in a document by iterating over all ciphertexts C_i, computing $C_i \oplus X$ and checking if the output is of the form $\langle S, PRF_{K_s}(S) \rangle$ for some S.

A security analysis of the scheme reveals that it leaks keyword position in the document. It is also susceptible to statistical attacks (e.g. after various searches, the words in the document can be inferred).

Curtmola et al. In 2006, two new schemes, for non-adaptive and adaptive adversaries, where proposed [22]. Each one is, respectively, at least IND-CKA1 and IND-CKA2 secure. The idea behind these was to use an *inverted index*, which is an index per distinct word in the database, not per document.

Non-Adaptive Scheme Considering a database with n documents, we define two data structures: an array A, where each position of the array contains a node $n_{i,j}$ which belongs to the linked list L_i associated to the keyword w_i; a look-up table T to identify the first node of a linked list associated with a keyword.

To build A, we start by constructing the nodes. These nodes are encrypted, stored in random order (using permutations) and augmented with information do find and decrypt the next node in the list. The augmentation step happens before the encryption of the node. Each node $n_{i,j} = \langle id_m, k_{next}, p_{next} \| \varnothing \rangle$, includes information about the id of document m ($0 \leq m \leq n$) which contains the keyword, a randomly generated key used to encrypt the next node and a pointer to the next node in the list, or $\varnothing$ if it is the last one. The randomized disposition of the nodes, hides the length of the lists.

The look-up table I associates each keyword w_i with the node $n_{i,0} = \langle k_{i,1}, p_{i,1} \rangle \oplus PRF_k(w_i)$ stored in a random position using a Pseudo-Random Permutation (PRP), $PRP_{k'}(w_i)$, where $k \neq k'$. This node contains the encryption key and the pointer to the first node of the linked list L_i.

To search, we generate trapdoor $T_w = \langle PRF_k(w), PRP_{k'}(w) \rangle$. The server uses the trapdoor to find the correct node $n_{i,0} = T[PRP_{k'}(w)]$ and then retrieve all the identifiers of the documents that contain the keyword.

Adaptive Scheme Considering a database with n documents, we now define only a look-up table I, but instead of generating the index for a single keyword w, we generate it for the family of the keyword. The family of a keyword is defined as a set of labels $F_w = \{w | str(j) : 1 \leq j \leq t\}$ where j is an integer converted to string, using str, t is the number of identifiers of documents containing w, and the label is the result of concatenating w with j. The look-up table associates each label with the identifier of a document containing the w, in a random position, using $PRP_{k'}(w_i | str(j))$. To hide differences in the number of labels per keyword, extra entries are padded.

We generate the trapdoor $T_w = (t_1, \cdots, tn) = (PRP_k(w \mid str(1)), \cdots, PRP_k(w \mid str(n)))$ to search. The server uses the trapdoor values to find valid index positions in I and outputs the set of identifiers.

Generalization to the Dynamic Setting The previous schemes where considered static, as they do not allow modifications on the database after setting up the scheme. In 2012, Kamara et al. [41] proposed a *dynamic* extension to the *non-adaptive* scheme in [22]. They used a *deletion* array to track the positions on the search array that needed to be modified,

after an update. They encrypted the array pointers, using homomorphic encryption, to support modifications on their values without decryption.

Generalization to Structured Data In 2010, Chase and Kamara [16] proposed a new adaptively secure scheme based on the *non-adaptive* scheme in [22]. This new scheme supports arbitrarily-structured data. Structured Symmetric Encryption is based on generating an inverted index in the form of a padded and permuted dictionary. They associated data items with metadata, allowing information about structure to be preserved if needed.

Generalization to Graph-Structured Data After STE, additional research has been conducted to support operations on graph-structured data.

Some authors extended STE to support queries on conceptual graphs [70], but they need to compute all possible query answers *a priori*.

Others, proposed a SSE scheme that supports *subgraph isomorphism queries* [13]. In their approach, they extract features from the graphs and generate the encrypted-indexes from the extracted features. This scheme was planned for generic graphs.

Finally, in [84] they combine an index-based SSE with a bucketization algorithm. By doing so, queries do not leak information about the structure of the graph. Although the authors state that their approach works with any kind of query, they only focused on neighbor and adjacency ones.

2.2.5 Secure Semantic Data Sharing

Secure collaborative solutions for semantic data development require secure sharing, access control and the execution of secure read and write operations over the encrypted data. If the dataset is an ontology the reasoning capabilities of the underlying system should not be impaired.

One of the simplest works in the field focuses solely on protecting individuals in OWL2 ontologies [58], uniquely encrypting their values and leaving the rest of the data in plaintext. This approach aimed at retaining functionality but leaks too much information. The following solutions try to achieve more robust security guaranties.

2.2.5.1 Self-Enforcing Access Control for Encrypted RDF

This work researched the implementation of access control in RDF datasets [28] through the use of functional encryption [9].

Encryption Scheme In the proposed encryption scheme, functions correspond to the computation of inner-products over $\mathbb{Z}_N$. The set of possible ciphertext attributes of length n is $\sum = \mathbb{Z}_N^n$ and the class of decryption key functions is $F = \{f_{\overrightarrow{x}} \mid \overrightarrow{x} \in \mathbb{Z}_N^n\}$. An attribute vector $\overrightarrow{y} \in F$ is associated with each ciphertext. A decryption key is a vector $\overrightarrow{x}$ such that $f_{\overrightarrow{x}}(y) = 1 \;\; iff \;\; \sum_{i=1}^{n} y_i x_i = 0$

To encrypt an RDF triple $t_i = (s_i, p_i, o_i)$ we functionally encrypt a random seed m_{t_i}, with an attribute vector $\overrightarrow{y}_{t_i} = (y_{s_i}, y'_{s_i}, y_{p_i}, y'_{p_i}, y_{o_i}, y'_{o_i})$ such that $\overrightarrow{y}_l = -r \cdot \sigma(l),\ y'_l := r,\ l \in \{s, p, o\}$ with σ being a function that maps a triple's subject, predicate and object to a value in $\mathbb{Z}_N$, and r a random value also in $\mathbb{Z}_N$

Advanced Encryption Standard (AES)[6] encryption is used to encrypt the plaintext triple using a key derivable from the generated seed m_{t_i}. The final ciphertext $c_{t_i} = \langle AES(t_i), FE(m_{t_i}) \rangle$ corresponds to both the AES ciphertext and the ciphertext of the seed.

The decryption keys are generated taking into account the possible triple patterns $tp = (s, p, o)$ combinations. For each triple pattern we define $\overrightarrow{x}_{tp} = (x_s, x'_s, x_p, x'_p, x_o, x'_o)$ such that $x_l := 1$, $x'_l := \sigma(l)$ if l is bound and $x_l := 0$, $x'_l := 0$ otherwise.

Finally, to decrypt a triple $t = (s_t, p_t, o_t)$, given a triple pattern $tp = (s_{tp}, p_{tp}, o_{tp})$ we compute the inner product $\overrightarrow{y}_t \cdot \overrightarrow{x}_{tp} = y_{s_t} x_{s_{tp}} + y'_{s_t} x'_{s_{tp}} + y_{p_t} x_{p_{tp}} + y'_{p_t} x'_{p_{tp}} + y_{o_t} x_{o_{tp}} + y'_{o_t} x'_{o_{tp}}$

Only if the result of the inner product is 0 can we decrypt the seed, derive the correct AES key and decrypt the triple.

Data Structure The data is stored in a key-value structure where the keys are unique integer IDs and the values correspond to the encrypted triples. The IDs are indexed using two strategies.

The first strategy is a *3-Index* approach. We compute secure hashes $h(s)$, $h(p)$ and $h(o)$ for each s, p, o (unbound values correspond to $h(?)$). Then, we generate three index tables *SPO*, *POS* and *OSP* where we store all triple pattern combinations and associate each value to the corresponding ID.

The second strategy is *vertical partitioning*. Instead of indexing the permutations of subjects, predicates and objects. We create one table per predicate, indexing the subject, object pairs that are related via that predicate. Each ID is indexed by a $("so"|"os", s, o)$.

2) *TODO:* Add image with both index schemes

T[2]

2.2.5.2 CryptGraphDB

CryptGraphDB [2] is an adaptation of the *CryptDB* [73, 72, 71] system to graph-structured data. It is capable of answering cypher [59] queries over conceptual graphs. We will start by describing the *CryptDB* architecture, security guaranties, query execution pipeline and encryption schemes. We will then explain the adaptations employed by *CryptGraphDB*.

System Architecture *CryptDB* is a practical solution for processing SQL queries over encrypted relational datasets. This solution leverages onion-encryption to support various primitive operations necessary to answer a SQL query (equality checks, order comparisons, aggregates and joins).

The system is composed by an application server, a proxy and an unmodified Database Management Systems (DBMS). The proxy intercepts all queries, encrypts and decrypts all data, and changes query operators while preserving the semantics of the original query.

3) *TODO:* Add image of Cryptdb system

T[3]

Security Threats *CryptDB* addresses two security threats. The first threat, consists in a passive attacker (e.g. database administrator), which only observes confidential data in the DBMS server. Against this attacker *CryptDB* provides a few security guaranties: sensitive data is always encrypted; the DBMS server cannot generate results for queries whose primitives do not belong in the subset of supported operations; information leakage depends on the types of computations that will be executed over the data - if no relational predicate filtering is needed, nothing more than the data's length is leaked; on

[6] For information on AES we recommend reading chapter 6 of [66].

equality checks, it reveals the histogram of encrypted values in the respective columns; in order checks, the system reveals the order between the encrypted values in the column.

The second threat is an attacker that gains control over the proxy, application server and DBMS server. Against this adversary, *CryptDB* provides confidentiality of data belonging to users not logged in during the period of the compromise (until it has been fixed). The adversary can, however, delete data stored in the database.

Besides confidentiality, *CryptDB* does not ensure integrity, freshness, or completeness of results.

Query Execution Pipeline For executing a query issued by the application server, a sequence of steps are necessary: i) the proxy intercepts and rewrites the query, anonymizing the table and column names and encrypting each constant present with the adequate encryption scheme for the desired operation, using a master key; ii) the proxy checks whether it should provide keys to the DBMS server, so that adjustments can be done on the encryption layers of the targeted data, if the check is positive it issues an UPDATE query that triggers a custom function to adjust the encryption of the selected columns; iii) the encrypted query is forwarded to the DBMS server and is executed using plain SQL; iv) the encrypted results are sent to the proxy, which are then decrypted and forwarded to the application server.

Adjustable Encryption Schemes To avoid unnecessary leakage of information data is encrypted with different layers, like an onion. When operations need encryption schemes with lower security guaranties, the data encryption layers are pealed. For each layer, per onion, per column, the proxy uses the same encryption key, derived from a master key, MK. This reduces the leakage of relations between columns and avoids the complexity of having different keys for each row in a column. For table t, column c, onion o, and encryption layer l, the proxy would use the encryption key $K_{t,c,o,l} = PRP_{MK}(t,c,o,l)$.

We now present the different encryption schemes supported by *CryptDB*:

- **Random (RND)** A probabilistic encryption scheme that provides the most security but does not allow any computations on the ciphertexts.
- **Deterministic (DET)** A deterministic encryption scheme which allows equality checks to be executed over ciphertexts.
- **Order-Preserving (OPE)** An encryption scheme that preserves order between the ciphertexts in a column. This allows the server to execute range queries.
- **Homomorphic encryption (HOM)** The Paillier cryptosystem [64] is used to probabilistically encrypt numeric values. This scheme allows additive operations to be executed with ciphertexts.
- **Join (equi-JOIN)** In *CryptDB* columns are encrypted with a different encryption keys. Essentially, joins can not be performed between columns until the encryption keys of the columns coincide. To support join operations between different columns a new primitive is introduced JOIN-ADJ. It can be interpreted as a *keyed random hash* with the property that the hashes can be adjusted to modify their key without access to the plaintext, using an adjustment key $\Delta K = k_{c_1}/k_{c_2}$. The adjustment of the columns is done only on one of the two (the first column in lexicographic order). The information leakage is similar to that of the DET layer.

- **OPE-JOIN** *CryptDB* does not provide a construct to dynamically modify the keys of columns encrypted with the OPE scheme. To support range joins, these columns have to be specified before, so that they are encrypted under the same key.

- **Word search (SEARCH)** This encryption scheme allows queries with the LIKE keyword. In *CryptDB* they used the scheme by Song et al. [81]. The information leakage is similar to that of the RND layer.

The following images provide, respectively, an overview of the different onions types in *CryptDB* and an example a query execution where layer adjustments are needed.

4) *TODO:* Add query flow image

T[4] T[5]

5) *TODO:* Add onion encryption image

CryptGraphDB constructs upon the approach of *CryptDB*, adapting the solution to be able to answer *cypher* queries [59]. The proxy is implemented to handle *cypher* queries instead of SQL queries, but keeps the same functionality. Furthermore, as the *cypher* language does not require the join operator, *CryptGraphDB* only maintains the RND, DET, OPE, HOM and SEARCH encryption schemes.

2.2.5.3 GOOSE

GOOSE [18] is a secure graph outsourcing framework capable of answering UCRPQ queries. The solution uses secure multiparty computation techniques, encryption schemes and a standard SPARQL engine.

Security Guaranties

System Architecture T[6]

6) *TODO:* Describe protocol. Describe security guaranties

2.2.5.4 HDT_{crypt}

The HDT_{crypt} [27] solution combines RDF compression with encryption. It is capable of efficient storage, but can only perform simple triple pattern search (joins and other complex operations need to be performed by the client).

T[7]

7) *TODO:* Describe successintly hdt tables, how to encrypt

2.2.5.5 Discussion

T[8] T[9] T[10] T[11] T[12]

8) *TODO:* Discuss trade-offs, remove from previous subsections. Do not forget to mention leakage that can arise from joins... Create resume table for solutions

For mitigating information leakage, *hybrid* SE schemes, that require *trusted hardware*, have been proposed [29], but solutions based on special hardware are not on the scope of this thesis.

Finally, there exists related research in specific security fields regarding RDF graphs.

One of those fields is ensuring *integrity* and *authenticity* of data. This has been widely studied with various signage schemes [14, 92, 42, 43, 44, 85] and one service-oriented approach [49] having been proposed.

9) *TODO:* 1 - hashes for indexes, leaked structure. Only search over simple patterns, no complex operations, no updates, only upload

10) *TODO:* 2 - Not for SPARQL, same leakage as CryptDB, can do updates

11) *TODO:* 3 - Leaks structure. Can answer complex queries, no updates

The other field is *access control*. Various schemes have been studied and most try to embed the data with the policies, using annotations. This allows for high-granularity in the access control, which can be inferred, but requires conflict resolution [40, 1, 30, 65, 19, 46, 20]. A more simple approach is token-based [45]. As they are not the main focus of this thesis, being only complementary work on the areas of security and ontology-engineering, we will not elaborate further on these.

2.3 Summary

In Section 2.1, we described the layered architecture of the *Semantic Web* and discussed the standard languages for describing its *data model* and *ontologies*. More specifically, we covered the semantics and syntax of RDF and explained the primitives of RDFS.

Additionally, we specified the structure of OWL2, focusing on the mechanism it provides to write ontologies. We also discussed the restriction imposed by *OWL DL* to remain decidable.

Furthermore, we wrote about SPARQL, discussing its mathematical basis, describing its syntax and enumerating its various types of queries. In an additional note, SPARQL has been widely studied in the literature. Research has been conducted on its semantics and complexity [68], on efficient processing of its queries [48] and on its patterns of use [10, 69]. As far as we know, no work has yet been done on its usage with encrypted RDF data.

We finished Section 2.1 by explaining tools and software that is used in this field of research.

In Section 2.2, we discussed various solutions to execute secure computations in untrusted servers. We started by describing the *honest-but-curious*, or *passive*, adversary model and explained why *Encryption at Rest* and ORAM are not practical approaches.

We proceeded to cover Homomorphic Encryption schemes, informally mentioning the mathematical basis of HE and enumerating the type of schemes available.

In Section 2.2.3, we explained the difference between SSE and PEKS. We then described the general model for index-based SE, as well as the *Client/Server* architecture of SE systems. We continued by discussing how to extend S/S architectures, and covered the types of privacy leakage that schemes typically have.

In Section 2.2.4 we specified the standard security definitions in use, and gave an in depth explanation of reference SSE schemes. We finished the SSE Section by showing recent generalizations, focusing on dynamic solutions and schemes that support structured data.

In Section 2.2.5, we mentioned related research on the topic of secure semantic data sharing, discussing the trade-off of each approach. Additionally, we briefly covered parallel work on security and ontologies in the fields of data *integrity*, *authenticity* and *access control*.

T[13] To conclude, we state that there still needs to be conducted more research in the field of SSE for graph-structured data. To justify our statement, we argue that approaches are either too generic [13], lack schema protection [58], have been implemented considering types of queries not used by SPARQL [84], or require pre-computation of all possible query answers beforehand [70].

13) *TODO:* re-contextualize based on new works

3

Approach to Elaboration

In this chapter we will present our approach to the elaboration phase. The remaining of the chapter is organized as follows: in Section 3.1, we introduce the proposed solution; in Section 3.2 we will define the system model and architecture; in Section 3.3 we present the expected testing and validation scenarios.

3.1 A Protocol for Secure Operations over Privacy-Enhanced Ontologies

As an answer to the lack of solutions for the problems exposed in Section 1.2, we propose a new protocol that extends upon existing SSE schemes for graph-structured data. This protocol will be designed taking into consideration the necessary privacy and system requirements that exist when working with OWL2 ontologies:

Privacy Requirements Ontologies are used in various different domains, as such, they might contain sensitive information. Both IND-CKA1 and IND-CKA2 solutions guarantee that trapdoors do not leak information about keywords (besides leakage from search and access patterns). However, IND-CKA1 only does so if trapdoors are generated all at once. This is not practical for our protocol. An appropriate solution must be a SSE scheme at least IND-CKA2 secure.

System Requirements The solution should be *transparent* and as *efficient* as possible. Working with encrypted ontologies must not affect a user's workflow in any significant way. Furthermore, it should also be *flexible* and *modular*, so that it supports future improvements or modifications if needed.

Finally, professionals must still be able to execute the majority of operations that they would be able to, with normal ontologies, even if they are working with encrypted ontologies:

- **P1 - Static Setting** In this setting the protocol will only support SPARQL queries, for read operations only (*SELECT*, *ASK*, *CONSTRUCT*, *DESCRIBE*). Focusing strictly on preserving query expressiveness.
- **P2 - Dynamic Setting** In this setting, the protocol will introduce support for SPARQL queries that modify the encrypted ontology.

3.2 System Model & Architecture

System Model There exist mechanisms to provide secure and reliable communication channels, (e.g. Transport Layer Security (TLS) [89], Hyper Text Transfer Protocol Secure (HTTPS) [38] and Transmission Control Protocol (TCP)) and which are adopted by trusted cloud providers. These provide security guarantees in transit (while data is being transferred from or loaded into servers). They do not, however, protect against an adversary who might have access to the servers where the data is stored.

The latter is the type of adversary which our protocol will target. Therefore, we will assume that all communication channels are *reliable* and *secure*. The system will follow the *client/server* model of SE schemes. More specifically, it will be, initially, a S/S solution, further extended to M/M using a *key sharing* approach. It will include a *data-owner*, *users*, an *untrusted server* and *adversaries*, as observed in figure 3.1:

- *Data-Owner* The data-owner is the entity that owns the ontology, originally. Therefore, it is the one responsible for generating the secret keys and secure-indexes, encrypting the ontology and uploading it to a remote server. It will also be responsible for sharing the secret keys with authorized users. Additionally, the data-owner must execute *user revocation* when needed, updating the secret keys and re-encrypting the ontology.

- *Users* The users are entities who have access to the ontology and have the right secret keys. This allows them to execute secure *SPARQL* queries, over the remote encrypted ontology.

- *Untrusted Server* The untrusted server is the machine where the encrypted ontology will be stored.

- *Adversaries* An adversary is an entity that has access to the untrusted servers or possesses secret keys but whose access to the ontology has been revoked. The adversaries follow the *honest-but-curious* model.

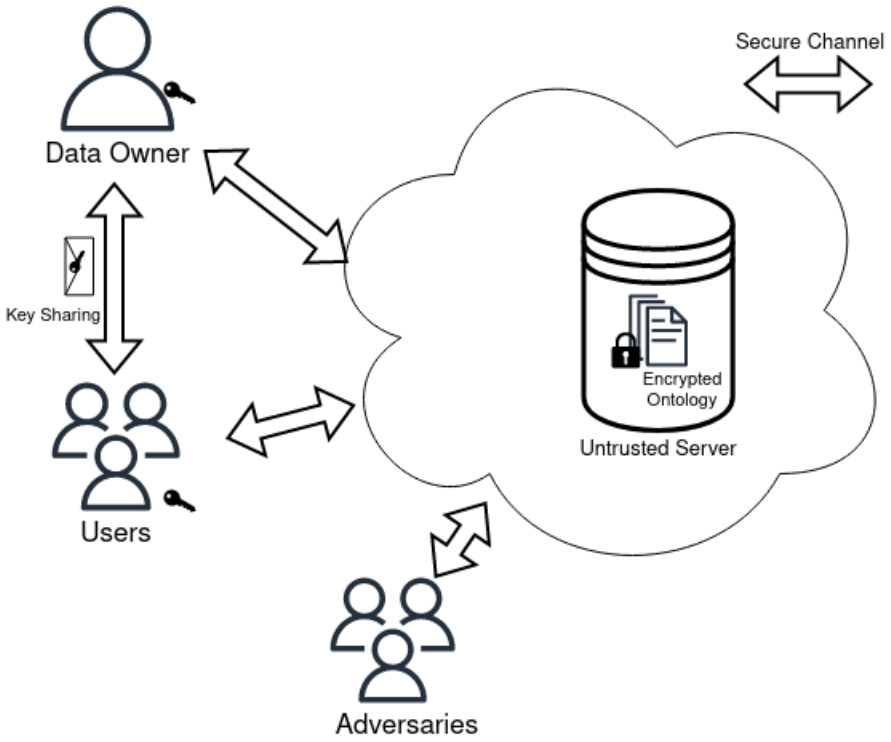

Figure 3.1: The System Model.

System Architecture At a macro level, the solution will be a distributed software solution, including a *client* and a *server* component. Besides the necessary communication modules, they will include:

- *Client* The modules to encrypt OWL2 ontologies, parse SPARQL queries and generate secret keys, encrypted indexes and trapdoors
- *Server* The modules to load the encrypted ontology and indexes, parse trapdoors and execute secure searches.

3.3 Evaluation Guidelines

After every implementation step we will proceed with evaluation of the prototype. The testing will follow the existing Lehigh University Benchmark [17], executing the protocol in two scenarios, with the same dataset. In the first, operations will be executed with the ontology in plaintext and, in the second, with the ontology encrypted.

For evaluation, we will check performance metrics (e.g. throughput, latency). Additionally, we will also evaluate the security guarantees of the solution.

For validation, we will compare whether the output of the benchmark, using the plaintext ontology, is the same to the one, using the encrypted ontology.

4

Work Plan

In this chapter we present the work plan for the elaboration phase (Figure 4.1). During the elaboration phase we will do the following tasks:

Protocol Design which consists in studying and formalizing the design of the SSE protocols to implement;

Implementation consists in the implementation of the protocols. At the end of this task we will have produced software artifacts;

Evaluation will be done at the end of each implementation task, and consists in the evaluation of the prototypes, in terms of performance and security. At the end of this task we will have produced graphs which describe the result metrics;

Validation will be done at the end of each implementation task, and consists in testing the correctness of the prototyped solutions. At the end of this task we will have produced graphs which describe the result metrics;

Writing the rest of thesis, which will be done in parallel with the other tasks.

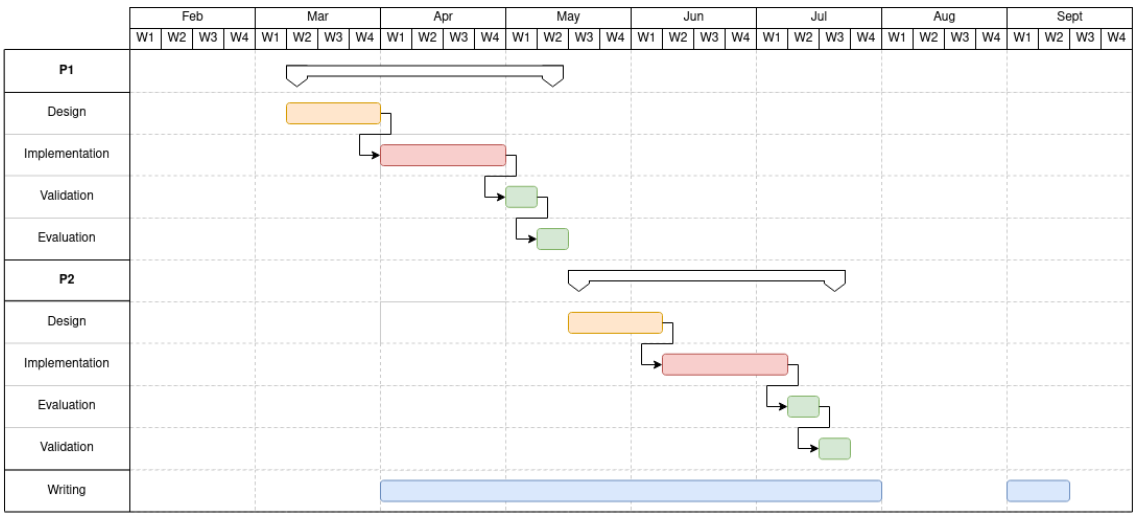

Figure 4.1: Work Plan

Bibliography

[1] F. Abel et al. "Enabling advanced and context-dependent access control in RDF stores". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 4825 LNCS. 2007, pp. 1–14. ISBN: 9783540762973. DOI: 10.1007/978-3-540-76298-0_1 (cit. on p. 28).

[2] N. Aburawi, A. Lisitsa, and F. Coenen. "Querying Encrypted Graph Databases". In: *ICISSP 2018 - Proceedings of the 4th International Conference on Information Systems Security and Privacy*. Vol. 2018-January. 2018. DOI: 10.5220/0006660004470451 (cit. on pp. 3, 26).

[3] A. Acar et al. "A survey on homomorphic encryption schemes: Theory and implementation". In: vol. 51. 2018. DOI: 10.1145/3214303 (cit. on pp. 3, 20).

[4] Amazon Web Services, Inc. or its affiliates. *Amazon Neptune*. https://docs.aws.amazon.com/neptune/index.html. Accessed: 2022-2-17 (cit. on p. 19).

[5] G. Antoniou et al. "A Semantic Web Primer (Third Edition)". In: *The MIT Press* (2012), p. 287 (cit. on pp. 6–8, 11, 12, 15).

[6] N. Ashish. "Semantic-Web Technology: Applications at NASA". In: *Dagstuhl Seminar Proceedings*. 2005 (cit. on p. 2).

[7] T. Berners-Lee, J. Hendler, and O. Lassila. *The semantic web*. Vol. 284. 2001. DOI: 10.1038/scientificamerican0501-34 (cit. on p. 1).

[8] M. Blaze, G. Bleumer, and M. Strauss. "Divertible protocols and atomic proxy cryptography". In: *Advances in Cryptology — EUROCRYPT'98*. Springer Berlin Heidelberg, 1998, pp. 127–144. DOI: 10.1007/BFb0054122 (cit. on p. 22).

[9] D. Boneh, A. Sahai, and B. Waters. "Functional Encryption: Definitions and Challenges". In: *Cryptology ePrint Archive* (2010) (cit. on p. 25).

[10] A. Bonifati, W. Martens, and T. Timm. "An Analytical Study of Large SPARQL Query Logs". In: (Aug. 2017). arXiv: 1708.00363 [cs.DB] (cit. on p. 29).

[11] C. Bösch et al. "A Survey of Provably Secure Searchable Encryption". In: *ACM Comput. Surv.* 47.2 (Aug. 2014), pp. 1–51. ISSN: 0360-0300. DOI: 10.1145/2636328 (cit. on pp. 2, 3, 20–22).

[12] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. "(Leveled) Fully Homomorphic Encryption without Bootstrapping". In: *ACM Trans. Comput. Theory* 6.3 (July 2014), pp. 1–36. ISSN: 1942-3454. DOI: 10.1145/2633600 (cit. on pp. 3, 21).

[13] N. Cao et al. "Privacy-preserving query over encrypted graph-structured data in cloud computing". In: *Proceedings - International Conference on Distributed Computing Systems*. 2011. DOI: 10.1109/ICDCS.2011.84 (cit. on pp. 25, 29).

[14] J. J. Carroll. "Signing RDF graphs". In: *Lect. Notes Comput. Sci.* 2870 (2003), pp. 369–384. ISSN: 0302-9743, 1611-3349. DOI: 10.1007/978-3-540-39718-2_24 (cit. on p. 28).

[15] M. Carroll, A. Van Der Merwe, and P. Kotzé. "Secure cloud computing: Benefits, risks and controls". In: *2011 Information Security for South Africa - Proceedings of the ISSA 2011 Conference*. 2011. DOI: 10.1109/ISSA.2011.6027519 (cit. on p. 2).

[16] M. Chase and S. Kamara. "Structured encryption and controlled disclosure". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 6477 LNCS. 2010. DOI: 10.1007/978-3-642-17373-8_33 (cit. on pp. 3, 20, 25).

[17] B. Chris and S. Andreas. *Berlin SPARQL Benchmark*. http://wbsg.informatik.uni-mannheim.de/bizer/berlinsparqlbenchmark/spec/index.html. Accessed: 2022-2-17 (cit. on pp. 19, 32).

[18] R. Ciucanu and P. Lafourcade. "GOOSE: A Secure Framework for Graph Outsourcing and SPARQL Evaluation". In: *Data and Applications Security and Privacy XXXIV*. Springer International Publishing, 2020, pp. 347–366. DOI: 10.1007/978-3-030-49669-2_20 (cit. on pp. 4, 28).

[19] L. Costabello, S. Villata, and F. Gandon. "Context-aware access control for RDF graph stores". In: *Frontiers in Artificial Intelligence and Applications*. Vol. 242. IOS Press, 2012, pp. 282–287. ISBN: 9781614990970. DOI: 10.3233/978-1-61499-098-7-282 (cit. on p. 28).

[20] L. Costabello et al. "Access control for HTTP operations on linked data". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 7882 LNCS. Springer Verlag, 2013, pp. 185–199. ISBN: 9783642382871. DOI: 10.1007/978-3-642-38288-8_13 (cit. on p. 28).

[21] O. Curé and G. Blin. *RDF Database Systems: Triples Storage and SPARQL Query Processing*. en. Morgan Kaufmann, Nov. 2014. ISBN: 9780128004708 (cit. on pp. 16, 17).

[22] R. Curtmola et al. "Searchable symmetric encryption: Improved definitions and efficient constructions". In: *J. Comput. Secur.* 19.5 (Nov. 2011), pp. 895–934. ISSN: 0926-227X, 1875-8924. DOI: 10.3233/jcs-2011-0426 (cit. on pp. 23–25).

[23] C. Dong, G. Russello, and N. Dulay. *Shared and Searchable Encrypted Data for Untrusted Servers*. 2008. DOI: 10.1007/978-3-540-70567-3_10 (cit. on p. 22).

[24] M. Dürst and M. Suignard. *Internationalized Resource Identifiers (IRIs)*. en. Tech. rep. 2005. DOI: 10.17487/RFC3987 (cit. on p. 6).

[25] T. Elgamal. "A public key cryptosystem and a signature scheme based on discrete logarithms". In: *IEEE Trans. Inf. Theory* 31.4 (July 1985), pp. 469–472. ISSN: 0018-9448, 1557-9654. DOI: 10.1109/TIT.1985.1057074 (cit. on p. 20).

[26] D. Evans, V. Kolesnikov, and M. Rosulek. "A Pragmatic Introduction to Secure Multi-Party Computation". In: *Foundations and Trends® in Privacy and Security* 2.2-3 (2018), pp. 70–246. ISSN: 2474-1558. DOI: 10.1561/3300000019 (cit. on pp. 2, 20).

[27] J. D. Fernández et al. "HDT crypt : Compression and encryption of RDF datasets". In: *Semant. Web* 11.2 (Feb. 2020), pp. 337–359. ISSN: 1570-0844, 2210-4968. DOI: 10.3233/sw-180335 (cit. on pp. 4, 28).

[28] J. D. Fernández et al. "Self-Enforcing Access Control for Encrypted RDF". In: *The Semantic Web*. Springer International Publishing, 2017, pp. 607–622. DOI: 10.1007/978-3-319-58068-5_37 (cit. on pp. 4, 25).

[29] B. Ferreira et al. "Boolean Searchable Symmetric Encryption with Filters on Trusted Hardware". In: *IEEE Trans. Dependable Secure Comput.* (2020), pp. 1–1. ISSN: 1941-0018. DOI: 10.1109/TDSC.2020.3012100 (cit. on pp. 3, 28).

[30] G. Flouris et al. "Controlling access to RDF graphs". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 6369 LNCS. 2010, pp. 107–117. ISBN: 9783642158766. DOI: 10.1007/978-3-642-15877-3_12 (cit. on p. 28).

[31] C. Fontaine and F. Galand. "A survey of homomorphic encryption for nonspecialists". In: *Eurasip Journal on Information Security* 2007 (2007). ISSN: 2510-523X. DOI: 10.1155/2007/13801 (cit. on pp. 3, 20).

[32] *Gartner forecasts worldwide public cloud end-user spending to reach nearly $600 billion in 2023*. en. https://www.gartner.com/en/newsroom/press-releases/2022-10-31-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-reach-nearly-600-billion-in-2023. Accessed: 2023-8-21 (cit. on p. 2).

[33] C. Gentry. "A Fully Homomorphic Encryption Scheme". PhD thesis. Stanford University, 2009 (cit. on p. 20).

[34] O. Goldreich. *Foundations of Cryptography: Volume 2, Basic Applications*. en. Cambridge University Press, 2001. ISBN: 9780521830843 (cit. on pp. 2, 20).

[35] O. Goldreich and R. Ostrovsky. "Software protection and simulation on oblivious RAMs". In: *J. ACM* 43.3 (May 1996), pp. 431–473. ISSN: 0004-5411. DOI: 10.1145/233551.233553 (cit. on pp. 3, 20).

[36] Y. Guo, Z. Pan, and J. Heflin. "LUBM: A benchmark for OWL knowledge base systems". In: *Journal of Web Semantics* 3.2 (Oct. 2005), pp. 158–182. ISSN: 1570-8268. DOI: 10.1016/j.websem.2005.06.005 (cit. on p. 19).

[37] M. A. Harris et al. "The Gene Ontology (GO) database and informatics resource". en. In: *Nucleic Acids Res.* 32.Database issue (Jan. 2004), pp. D258–61. ISSN: 0305-1048, 1362-4962. DOI: 10.1093/nar/gkh036 (cit. on p. 1).

[38] *HTTP Over TLS*. en. https://datatracker.ietf.org/doc/html/rfc2818. Accessed: 2022-2-18 (cit. on p. 31).

[39] A.-A. Ivan and Y. Dodis. "Proxy Cryptography Revisited". In: *NDSS*. 2003 (cit. on p. 22).

[40] A. Jain and C. Farkas. "Secure resource description framework: An access control model". In: *Proceedings of ACM Symposium on Access Control Models and Technologies, SACMAT*. Vol. 2006. 2006, pp. 121–129. ISBN: 9781595933546 (cit. on p. 28).

[41] S. Kamara, C. Papamanthou, and T. Roeder. "Dynamic searchable symmetric encryption". In: *Proceedings of the 2012 ACM conference on Computer and communications security*. CCS '12. Raleigh, North Carolina, USA: Association for Computing Machinery, Oct. 2012, pp. 965–976. ISBN: 9781450316514. DOI: 10.1145/2382196.2382298 (cit. on p. 24).

[42] A. Kasten and A. Scherp. *Iterative Signing of RDF(S) Graphs, Named Graphs, and OWL Graphs: Formalization and Application*. Tech. rep. Mainz: Universität Koblenz-Landau, Mar. 2013 (cit. on p. 28).

[43] A. Kasten and A. Scherp. "Towards a configurable framework for iterative signing of distributed graph data". In: *CEUR Workshop Proceedings*. Vol. 1121. 2014 (cit. on p. 28).

[44] A. Kasten, A. Scherp, and P. Schauß. "A framework for iterative signing of graph data on the web". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 8465 LNCS. Springer Verlag, 2014, pp. 146–160. ISBN: 9783319074429. DOI: 10.1007/978-3-319-07443-6_11 (cit. on p. 28).

[45] A. Khaled et al. "A token-based access control system for RDF data in the clouds". In: *Proceedings - 2nd IEEE International Conference on Cloud Computing Technology and Science, CloudCom 2010*. 2010, pp. 104–111. ISBN: 9780769543024. DOI: 10.1109/CloudCom.2010.76 (cit. on p. 28).

[46] S. Kirrane et al. "Secure manipulation of linked data". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 8218 LNCS. 2013. DOI: 10.1007/978-3-642-41335-3_16 (cit. on p. 28).

[47] G. Ladwig. *Efficient Optimization and Processing of Queries Over Text-rich Graph-structured Data*. en. KIT Scientific Publishing, 2013. ISBN: 9783731500155. DOI: 10.5445/KSP/1000034423 (cit. on p. 17).

[48] A. Letelier et al. "Static analysis and optimization of semantic web queries". In: *ACM Trans. Database Syst.* 38.4 (Dec. 2013), pp. 1–45. ISSN: 0362-5915. DOI: 10.1145/2500130 (cit. on pp. 17, 29).

[49] A. Maña, G. Pujol, and C. Pandolfo. "A distributed secure ontology for certified services oriented applications". In: *Proceedings - 5th IEEE International Conference on Semantic Computing, ICSC 2011*. 2011. DOI: 10.1109/ICSC.2011.105 (cit. on p. 28).

[50] MarkLogic. *MarkLogic server - multi-model database*. en. https://www.marklogic.com/product/marklogic-database-overview/. Accessed: 2022-2-17. Feb. 2020 (cit. on p. 19).

[51] J. P. McCrae. *The linked open data cloud*. en. https://lod-cloud.net/. Accessed: 2022-2-8 (cit. on p. 2).

[52] B. D. McKay and A. Piperno. "Practical graph isomorphism, II". In: *J. Symbolic Comput.* 60 (Jan. 2014), pp. 94–112. ISSN: 0747-7171. DOI: 10.1016/j.jsc.2013.09.003 (cit. on p. 17).

[53] G. A. Miller. "WordNet: a lexical database for English". In: *Commun. ACM* 38.11 (Nov. 1995), pp. 39–41. ISSN: 0001-0782. DOI: 10.1145/219717.219748 (cit. on p. 1).

[54] G. E. Modoni, M. Sacco, and W. Terkaj. "A survey of RDF store solutions". In: *2014 International Conference on Engineering, Technology and Innovation: Engineering Responsible Innovation in Products and Services, ICE 2014*. 2014. DOI: 10.1109/ICE.2014.6871541 (cit. on p. 16).

[55] *MongoDB Manual: Queryable encryption*. en. https://www.mongodb.com/docs/v7.0/core/queryable-encryption/. Accessed: 2023-8-21 (cit. on p. 2).

[56] M. A. Musen and Protégé Team. "The Protégé Project: A Look Back and a Look Forward". en. In: *AI Matters* 1.4 (June 2015), pp. 4–12. ISSN: 2372-3483. DOI: 10.1145/2757001.2757003 (cit. on pp. 2, 19).

[57] R. Neches et al. "Enabling technology for knowledge sharing". In: *AI Magazine* 12.3 (1991). ISSN: 0738-4602 (cit. on p. 1).

[58] R. Nejjahi, A. Marzouk, and N. Gherabi. "An Enhanced Method for Web Ontologies Encryption". In: *European Journal of Electrical Engineering and Computer Science* 3.4 (July 2019). ISSN: 2506-9853. DOI: 10.24018/ejece.2019.3.4.99 (cit. on pp. 25, 29).

[59] Neo4j, Inc. *Cypher Query Language*. en. https://neo4j.com/developer/cypher/. Accessed: 2023-8-24 (cit. on pp. 3, 26, 28).

[60] N. Noy et al. "Industry-scale knowledge graphs: Lessons and challenges". In: *Commun. ACM* 62.8 (2019). ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3331166 (cit. on p. 1).

[61] Ontotext. *GraphDB*. en. https://graphdb.ontotext.com/documentation/. Accessed: 2022-2-17 (cit. on p. 19).

[62] OpenLink Software. *Virtuoso Universal Server*. https://virtuoso.openlinksw.com/. Accessed: 2022-2-17 (cit. on p. 19).

[63] owl.cs. *OWL API repository*. en. https://github.com/owlcs/owlapi. Accessed: 2022-2-17 (cit. on p. 19).

[64] P. Paillier. "Public-Key Cryptosystems Based on Composite Degree Residuosity Classes". In: *Advances in Cryptology — EUROCRYPT '99*. Springer Berlin Heidelberg, 1999, pp. 223–238. DOI: 10.1007/3-540-48910-X_16 (cit. on pp. 20, 27).

[65] V. Papakonstantinou et al. "Access control for RDF graphs using abstract models". In: *Proceedings of ACM Symposium on Access Control Models and Technologies, SACMAT*. 2012. DOI: 10.1145/2295136.2295155 (cit. on p. 28).

[66] W. Pearson. *Cryptography and Network Security: Principles and Practice, Global Edition*. 7th ed. Pearson Education, 2017. ISBN: 9781292158587 (cit. on pp. 3, 23, 26).

[67] Z. Peng, S. Tang, and L. Jiang. *A Symmetric Authenticated Proxy Re-encryption Scheme with Provable Security*. 2017. DOI: 10.1007/978-3-319-68542-7_8 (cit. on p. 22).

[68] J. Pérez, M. Arenas, and C. Gutierrez. "Semantics and Complexity of SPARQL". In: *Semantics and complexity of SPARQL. ACM Trans. Database Syst* 34.16 (2009). DOI: 10.1145/1567274.1567278 (cit. on p. 29).

[69] F. Picalausa and S. Vansummeren. "What are real SPARQL queries like?" In: *Proceedings of the International Workshop on Semantic Web Information Management*. SWIM '11 Article 7. Athens, Greece: Association for Computing Machinery, June 2011, pp. 1–6. ISBN: 9781450306515. DOI: 10.1145/1999299.1999306 (cit. on pp. 16–18, 29).

[70] G. S. Poh, M. S. Mohamad, and M. R. Z'Aba. "Structured encryption for conceptual graphs". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 7631 LNCS. Springer Verlag, 2012, pp. 105–122. ISBN: 9783642341168. DOI: 10.1007/978-3-642-34117-5_7 (cit. on pp. 3, 25, 29).

[71] R. A. Popa, N. Zeldovich, and H. Balakrishnan. "CryptDB : A Practical Encrypted Relational DBMS". In: *Designfax* (2011). ISSN: 0163-6669 (cit. on pp. 3, 26).

[72] R. A. Popa et al. "CryptDB: Processing queries on an encrypted database". In: *Communications of the ACM*. Vol. 55. 2012. DOI: 10.1145/2330667.2330691 (cit. on pp. 3, 21, 26).

[73] R. A. Popa et al. "CryptDB: Protecting confidentiality with encrypted query processing". In: *SOSP'11 - Proceedings of the 23rd ACM Symposium on Operating Systems Principles*. 2011. DOI: 10.1145/2043556.2043566 (cit. on pp. 3, 26).

[74] *RDF 1.1: On semantics of RDF datasets*. en. https://www.w3.org/TR/rdf11-datasets/. Accessed: 2023-10-14 (cit. on p. 16).

[75] R. L. Rivest, L. Adleman, and M. L. Dertouzos. "On data banks and privacy homomorphisms". In: *Foundations of secure* (1978) (cit. on p. 20).

[76] R. L. Rivest, A. Shamir, and L. Adleman. "A method for obtaining digital signatures and public-key cryptosystems". en. In: *Commun. ACM* 21.2 (Feb. 1978), pp. 120–126. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/359340.359342 (cit. on p. 20).

[77] K. Sakurai, T. Nishide, and A. Syalim. *Improved proxy re-encryption scheme for symmetric key cryptography*. 2017. DOI: 10.1109/iwbis.2017.8275110 (cit. on p. 22).

[78] Security Compass. *The 2021 State of Cloud Adoption*. en. https://resources.securitycompass.com/reports/2021-state-of-cloud-adoption. Accessed: 2022-2-9. Sept. 2021 (cit. on p. 2).

[79] N. Sioutos et al. "NCI Thesaurus: a semantic model integrating cancer-related clinical and molecular information". en. In: *J. Biomed. Inform.* 40.1 (Feb. 2007), pp. 30–43. ISSN: 1532-0464, 1532-0480. DOI: 10.1016/j.jbi.2006.02.013 (cit. on p. 1).

[80] E. Sirin et al. "Pellet: A practical OWL-DL reasoner". In: *Journal of Web Semantics* 5.2 (June 2007), pp. 51–53. ISSN: 1570-8268. DOI: 10.1016/j.websem.2007.03.004 (cit. on p. 19).

[81] D. X. Song, D. Wagner, and A. Perrig. "Practical techniques for searches on encrypted data". In: *Proceeding 2000 IEEE Symposium on Security and Privacy. S P 2000*. May 2000, pp. 44–55. DOI: 10.1109/SECPRI.2000.848445 (cit. on pp. 3, 21, 28).

[82] D. X. Song, D. Wagner, and A. Perrig. "Practical techniques for searches on encrypted data". In: *Proceeding 2000 IEEE Symposium on Security and Privacy. S P 2000*. May 2000, pp. 44–55. DOI: 10.1109/SECPRI.2000.848445 (cit. on p. 23).

[83] R. Studer, V. R. Benjamins, and D. Fensel. "Knowledge engineering: Principles and methods". In: *Data Knowl. Eng.* 25.1 (Mar. 1998), pp. 161–197. ISSN: 0169-023X. DOI: 10.1016/S0169-023X(97)00056-6 (cit. on p. 11).

[84] G. Suntaxi, A. A. El Ghazi, and K. Böhm. "Secrecy and performance models for query processing on outsourced graph data". In: *Distributed and Parallel Databases* 39.1 (Mar. 2021), pp. 35–77. ISSN: 1573-7578. DOI: 10.1007/s10619-020-07284-0 (cit. on pp. 25, 29).

[85] A. Sutton and R. Samavi. "Timestamp-based Integrity Proofs for Linked Data". In: *Proceedings of the International Workshop on Semantic Big Data, SBD 2018 - In conjunction with the 2018 ACM SIGMOD/PODS Conference*. Association for Computing Machinery, Inc, June 2018. ISBN: 9781450357791. DOI: 10.1145/3208352.3208353 (cit. on p. 28).

[86] A. Syalim, T. Nishide, and K. Sakurai. "Realizing Proxy Re-encryption in the Symmetric World". In: *Informatics Engineering and Information Science*. Springer Berlin Heidelberg, 2011, pp. 259–274. DOI: 10.1007/978-3-642-25327-0_23 (cit. on p. 22).

[87] The Apache Software Foundation. *Apache Jena*. en. https://jena.apache.org/. Accessed: 2022-2-17 (cit. on p. 19).

[88] The Apache Software Foundation. *Apache Jena - TDB*. en. https://jena.apache.org/documentation/tdb/index.html. Accessed: 2022-2-17 (cit. on p. 19).

[89] *The Transport Layer Security (TLS) Protocol Version 1.3*. en. https://datatracker.ietf.org/doc/html/rfc8446. Accessed: 2022-2-18 (cit. on p. 31).

[90] T. Tudorache, J. Vendetti, and N. F. Noy. "Web-Protégé: A lightweight OWL ontology editor for the web". In: *CEUR Workshop Proceedings*. Vol. 432. 2009 (cit. on pp. 2, 19).

[91] T. Tudorache et al. "Supporting collaborative ontology development in Protégé". In: *Lect. Notes Comput. Sci.* 5318 LNCS (2008), pp. 17–32. ISSN: 0302-9743, 1611-3349. DOI: 10.1007/978-3-540-88564-1_2 (cit. on pp. 2, 19).

[92] G. Tummarello et al. "Signing individual fragments of an RDF graph". In: *14th International World Wide Web Conference, WWW2005*. 2005, pp. 1020–1021. ISBN: 9781595930514. DOI: 10.1145/1062745.1062848 (cit. on p. 28).

[93] W3C. *Extensible markup language (XML) 1.0 (fifth edition)*. en. https://www.w3.org/TR/xml/. Accessed: 2022-2-11 (cit. on p. 7).

[94] W3C. *OWL 2 web ontology language direct semantics (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-direct-semantics-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[95] W3C. *OWL 2 web ontology language document overview (second edition)*. en. https://www.w3.org/TR/owl2-overview/. Accessed: 2022-2-11 (cit. on pp. vi, 1, 7, 11, 12).

[96] W3C. *OWL 2 web ontology language Manchester syntax (second edition)*. en. https://www.w3.org/TR/2012/NOTE-owl2-manchester-syntax-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[97] W3C. *OWL 2 web ontology language profiles (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-profiles-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[98] W3C. *OWL 2 web ontology language quick reference guide (second edition)*. en. https://www.w3.org/TR/owl2-quick-reference/. Accessed: 2022-2-14 (cit. on p. 12).

[99] W3C. *OWL 2 web ontology language RDF-based semantics (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-rdf-based-semantics-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[100] W3C. *OWL 2 web ontology language structural specification and functional-style syntax (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-syntax-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[101] W3C. *OWL 2 web ontology language XML serialization (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-xml-serialization-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[102] W3C. *RDF 1.1 concepts and abstract syntax*. en. https://www.w3.org/TR/rdf11-concepts/. Accessed: 2022-2-11 (cit. on pp. 1, 7, 8).

[103] W3C. *RDF 1.1 TriG*. en. https://www.w3.org/TR/trig/. Accessed: 2022-2-13 (cit. on p. 8).

[104] W3C. *RDF 1.1 turtle*. en. https://www.w3.org/TR/turtle/. Accessed: 2022-2-13 (cit. on p. 8).

[105] W3C. *RDF 1.1 XML syntax*. en. https://www.w3.org/TR/rdf-syntax-grammar/. Accessed: 2022-2-13 (cit. on p. 9).

[106] W3C. *RDF Schema 1.1*. en. https://www.w3.org/TR/rdf-schema/. Accessed: 2022-2-11 (cit. on pp. 1, 7, 9).

[107] W3C. *RDFa core 1.1 - third edition*. en. https://www.w3.org/TR/rdfa-core/. Accessed: 2022-2-13 (cit. on p. 9).

[108] W3C. *Semantic Web Case Studies and Use Cases*. en. https://www.w3.org/2001/sw/sweo/public/UseCases/. Accessed: 2022-2-8 (cit. on p. 1).

[109] W3C. *SPARQL 1.1 graph store HTTP protocol*. en. https://www.w3.org/TR/sparql11-http-rdf-update/. Accessed: 2022-2-4. Mar. 2013 (cit. on p. 16).

[110] W3C. *SPARQL 1.1 Overview*. https://www.w3.org/TR/sparql11-overview/. Mar. 2013 (cit. on pp. 1, 16).

[111] W3C. *SPARQL 1.1 update*. en. https://www.w3.org/TR/sparql11-update/. Accessed: 2022-2-15 (cit. on pp. 1, 16, 19).

[112] W3C. *SPARQL Query Language for RDF*. https://www.w3.org/TR/rdf-sparql-query/. Jan. 2008 (cit. on pp. 1, 16, 18).

[113] W3C. *SWRL: A Semantic Web Rule Language Combining OWL and RuleML*. https://www.w3.org/Submission/SWRL/. May 2004 (cit. on p. 7).

[114] Wikipedia contributors. *Semantic web stack*. https://en.wikipedia.org/wiki/File:Semantic_web_stack.svg. Accessed: 2022-2-15 (cit. on pp. vi, 7).